



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

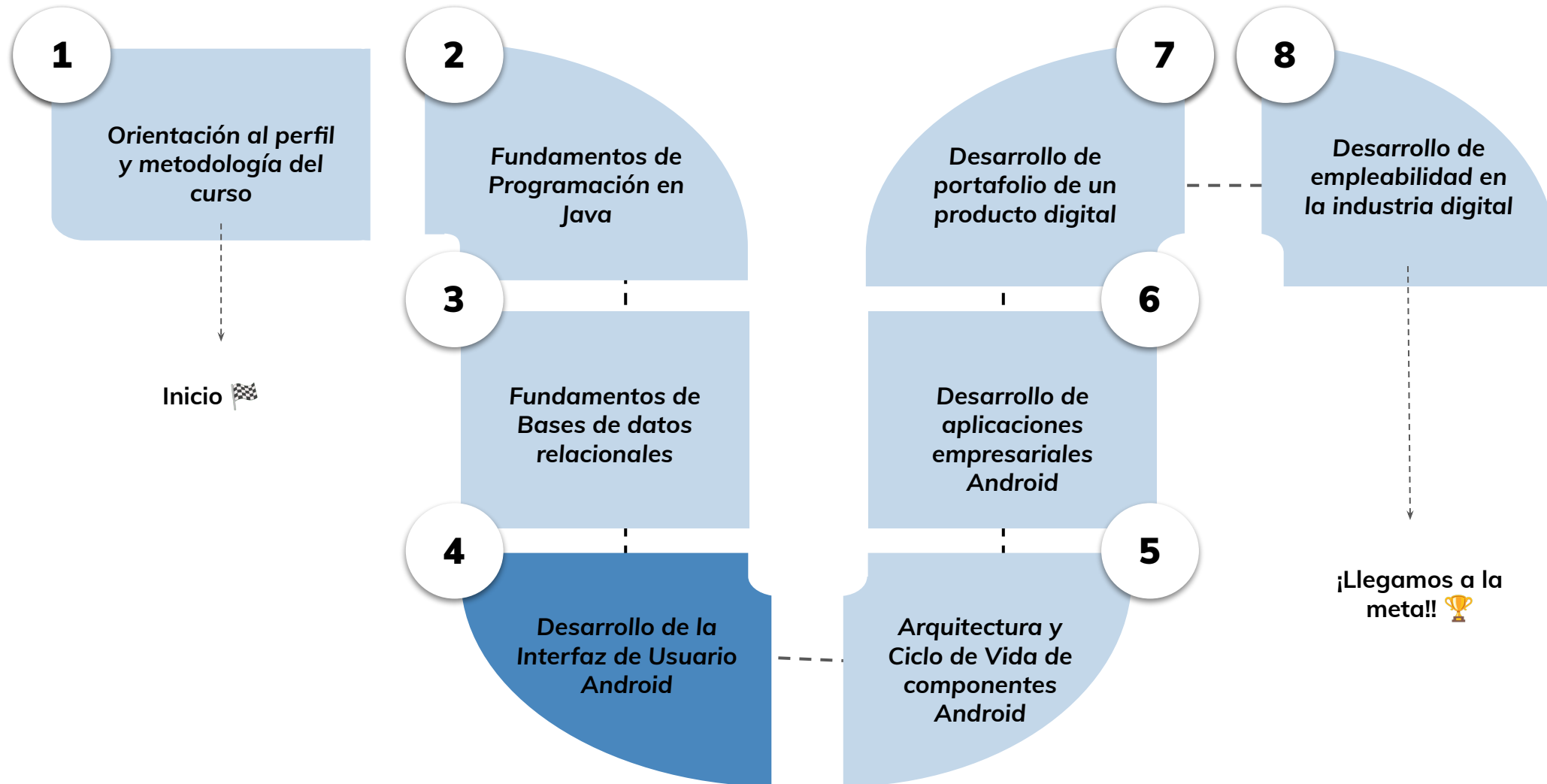


➤ Elementos de la interfaz de usuario

AE4.1: Utilizar elementos de interfaz de usuario básicos del entorno android para la implementación de un prototipo de acuerdo a las especificaciones entregadas.

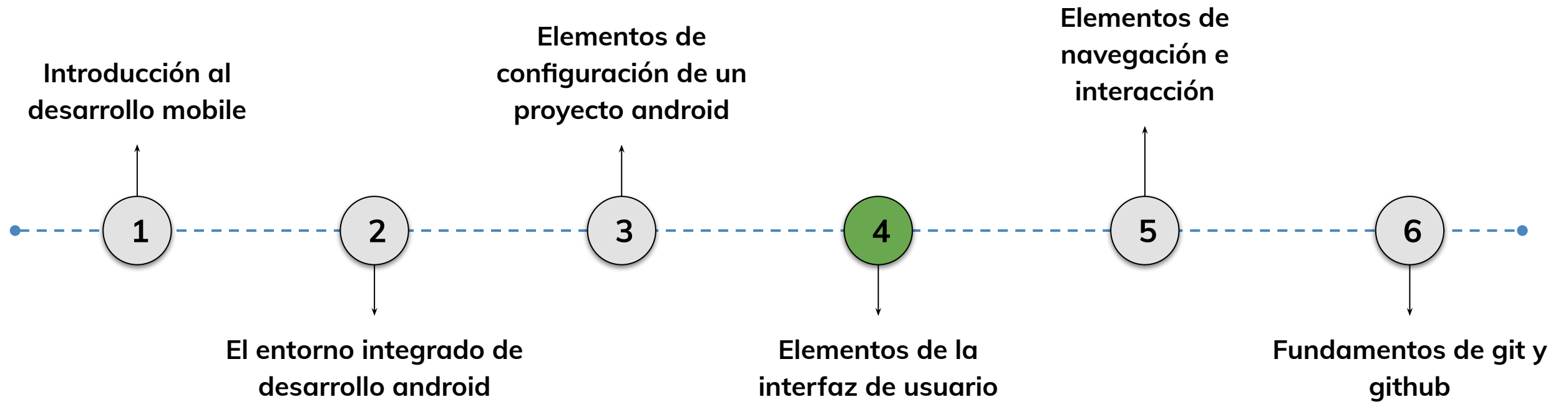
Hoja de ruta

¿Cuáles **módulos** conforman el programa?



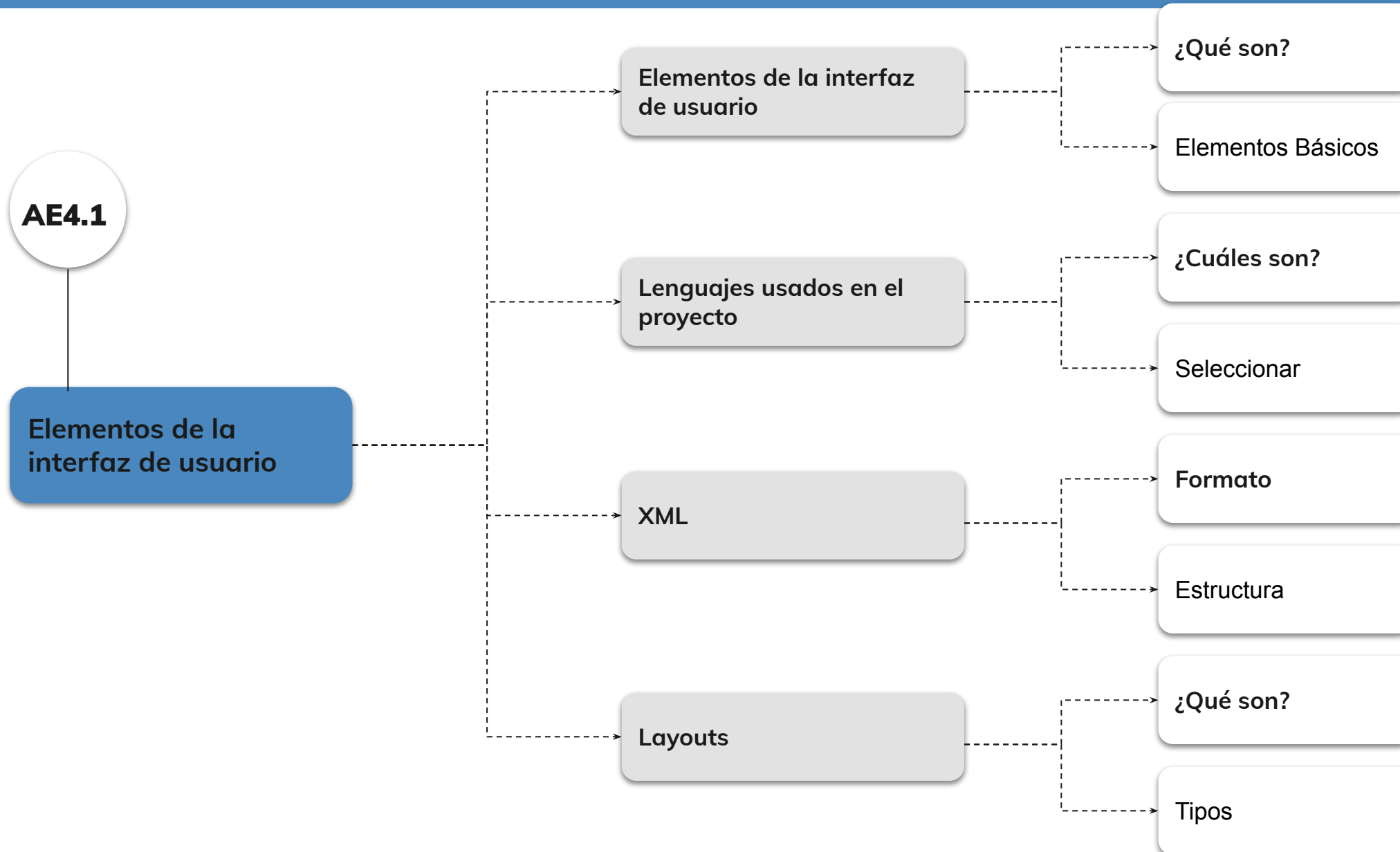
Roadmap de lecciones

¿Cuáles lecciones estaremos estudiando en este módulo?



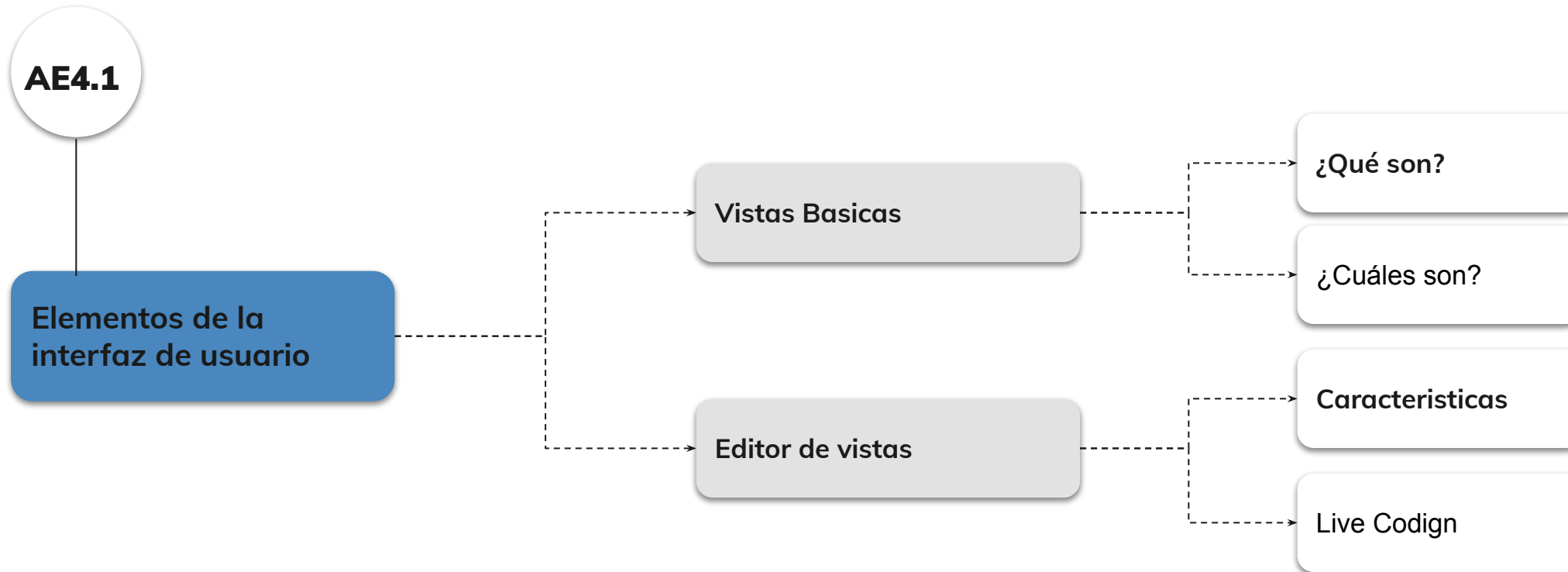
Learning Path

¿Cuáles temas trabajaremos hoy?



Learning Path

¿Cuáles temas trabajaremos hoy?





Objetivos de aprendizaje

¿Qué aprenderás?



- Utilizar componentes de un proyecto Android
- Distinguir los tipos de lenguajes usados en el proyecto, y su funcionalidad
- Entender sus usos y lugares
- Entender cómo se trabaja con el lenguaje XML en vista de Android
- Comprender el formato y reglas del lenguaje para su uso en creación de vistas.
- Saber más sobre cuáles son los casos de uso de los diferentes layouts disponibles
- Comprender cuáles son las vistas básicas que podemos usar en Android
- Saber más sobre las vistas básicas y sus aplicaciones
- Conocer el editor de vistas de Android Studio y sus funciones

< > Rompehielo

Contexto: 🙌

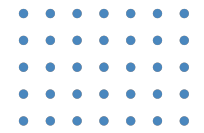
Cuando creamos aplicaciones en Android no solo necesitamos implementar la lógica de cómo queremos que se comporte si no también crear las vistas que el usuario va a usar para interactuar con la aplicación.

Consigna: ✍️

¿Cuál creen que va a ser el mayor desafío al construir estas vistas?



Elementos de la interfaz de usuario



Elementos de la interfaz de usuario

¿Qué son?:

Cuando vamos a definir los elementos de la interfaz de usuario tenemos que hablar de 2 grupos importantes: Las vistas y los contenedores de vistas.

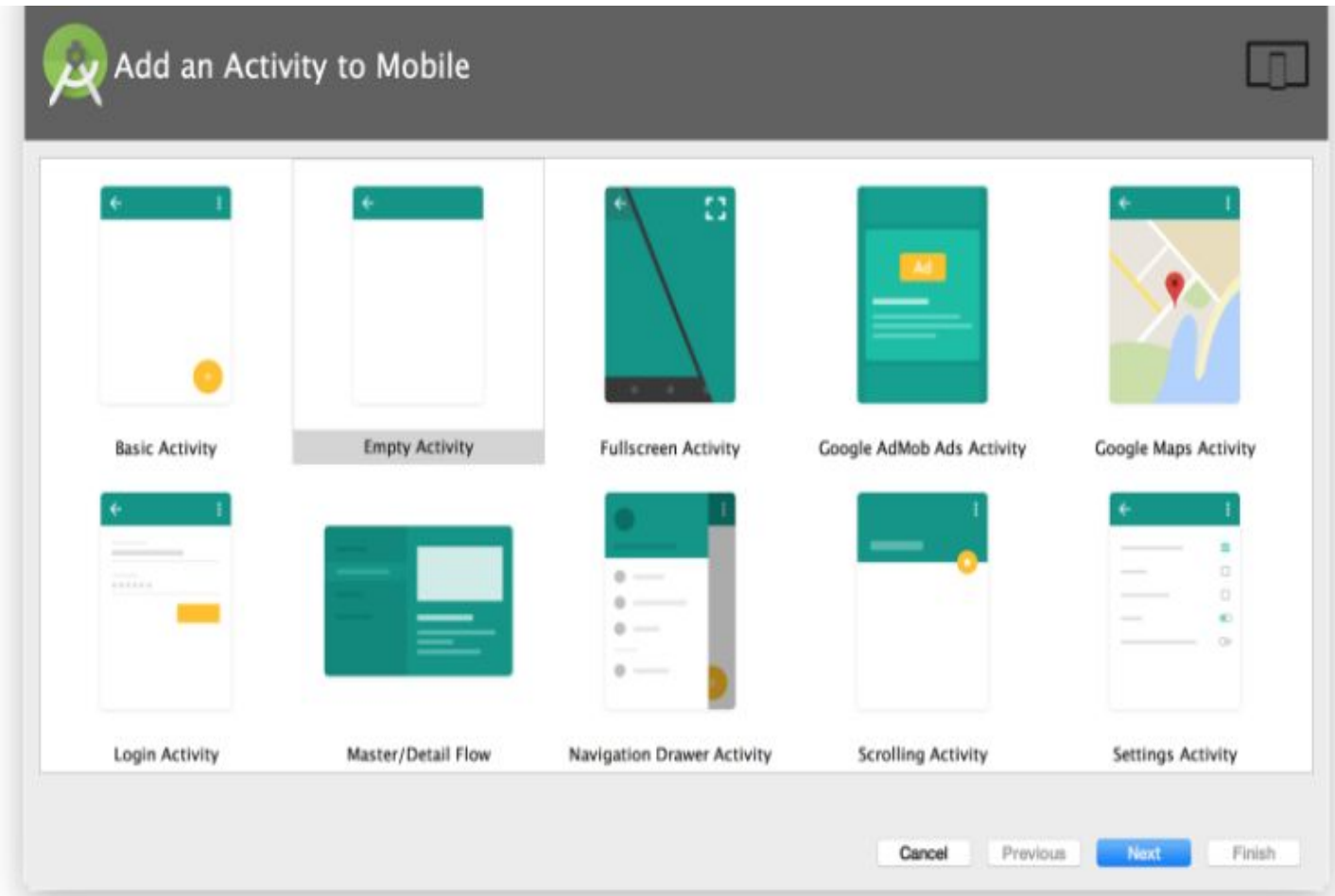
Las vistas son los componentes de interfaz de usuario más básicos, como botones, campos de texto, etiquetas, imágenes y otros elementos que los usuarios pueden ver y con los que pueden interactuar.

Por otro lado, los contenedores de vistas son los elementos que no tienen su propia visualización pero contienen configuraciones, formatos y todo lo necesario para mostrar otras vistas como serían los Fragmentos y las Actividades

Actividad

¿Qué son?:

Las actividades son componentes principales de la interfaz de usuario ya que representan en sí una pantalla. Una aplicación tiene que poseer como mínimo una Actividad para ser usable y son el más común contenedor de vistas. Tienen una parte visual y una parte lógica.



Vistas (Views)

¿Qué son?:

Las vistas son componentes visuales que se utilizan para construir la interfaz de usuario de una actividad. Algunos ejemplos de vistas incluyen TextView (para mostrar texto), ImageView (para mostrar imágenes), EditText (para ingresar texto), Button (para interactuar con el usuario), y muchas otras.



Layouts

¿Qué son?:

Los layouts son contenedores que organizan y controlan la disposición de las vistas en una actividad. Algunos ejemplos de layouts comunes incluyen LinearLayout, RelativeLayout, ConstraintLayout y FrameLayout.

Types of Android Layouts



StackLayout



AbsoluteLayout



RelativeLayout



GridLayout



ContentView



ScrollView

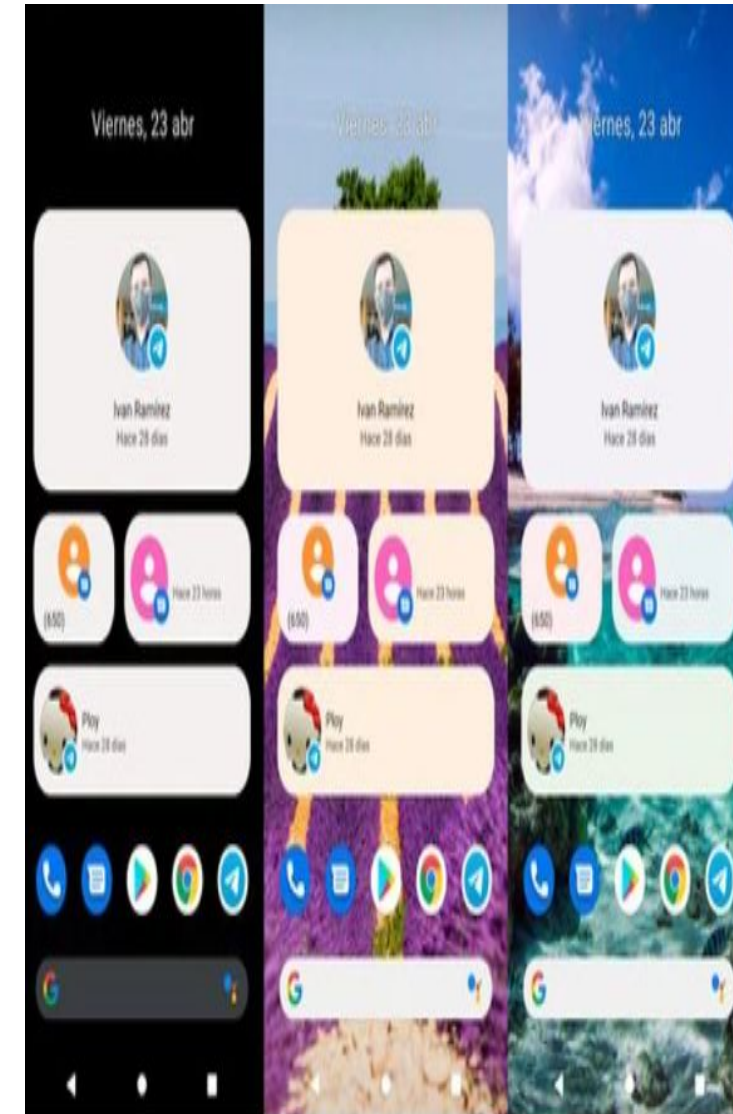


Frame

Widgets

¿Qué son?:

Los widgets son elementos interactivos que permiten a los usuarios interactuar con la aplicación. Esto incluye botones, cuadros de texto, casillas de verificación, botones de opción y barras de desplazamiento.



Menús

¿Qué son?:

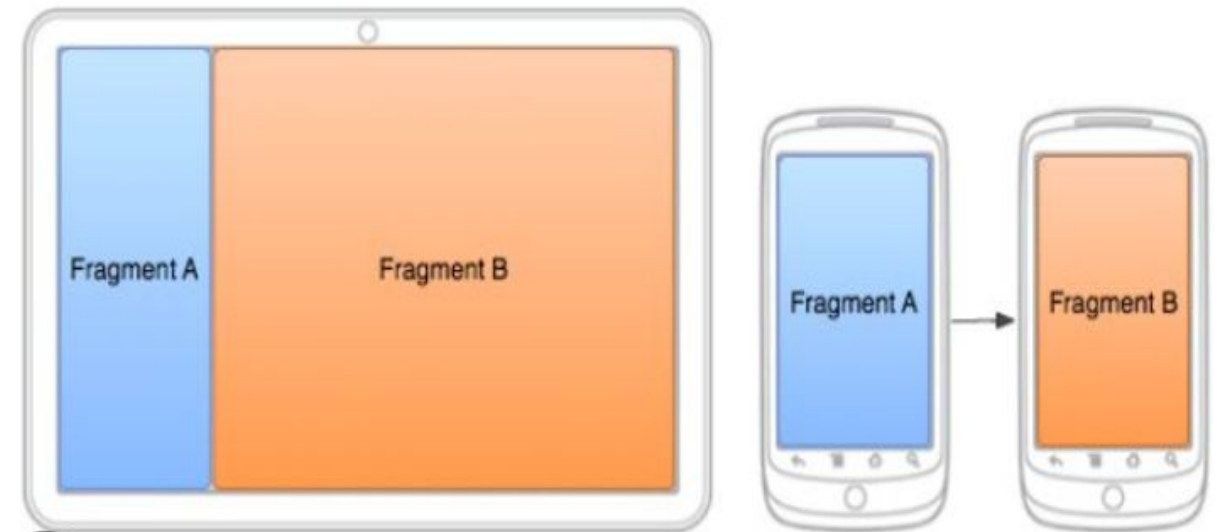
Los menús son una parte esencial de la interfaz de usuario en Android. Pueden ser menús de opciones en la barra de acción, menús emergentes o elementos de menú dentro de actividades.



Fragments

¿Qué son?:

Los fragmentos son componentes que permiten crear interfaces de usuario más flexibles y reutilizables. Puedes combinar fragmentos en actividades para crear interfaces más complejas. Son básicamente Actividades reutilizables pero sin tantas limitaciones y se pueden utilizar múltiples Fragmentos en una sola actividad.



LIVE CODING

Crear una actividad:

Vamos a crear un proyecto donde vamos mostrar una simple actividad:

1. Crear un nuevo proyecto
2. Seleccionar una actividad vacía
3. Darle cualquier nombre
4. Por ahora vamos a usar Java como lenguaje
5. Agregar vistas como botones y campos de texto
6. Agregar nombre a las vistas
7. Agregar textos y Menús en forma gráfica y en código.

Lenguajes usados



Lenguajes usados

¿Cuáles son?:

Cuando trabajamos con aplicaciones Android vamos a requerir usar varios lenguajes para crear diferentes partes de nuestro programa, Algunos de estos lenguajes van a ser usando para declarar como queremos que se vean nuestras pantallas y otros son usados para darle una lógica a lo que estamos creando. Vamos a hablar de varios de los posibles lenguajes que se usan en Android:

Java

¿Qué es?:

Java ha sido durante mucho tiempo el lenguaje de programación principal para el desarrollo de aplicaciones Android. La mayoría de las aplicaciones Android se han escrito en Java históricamente.

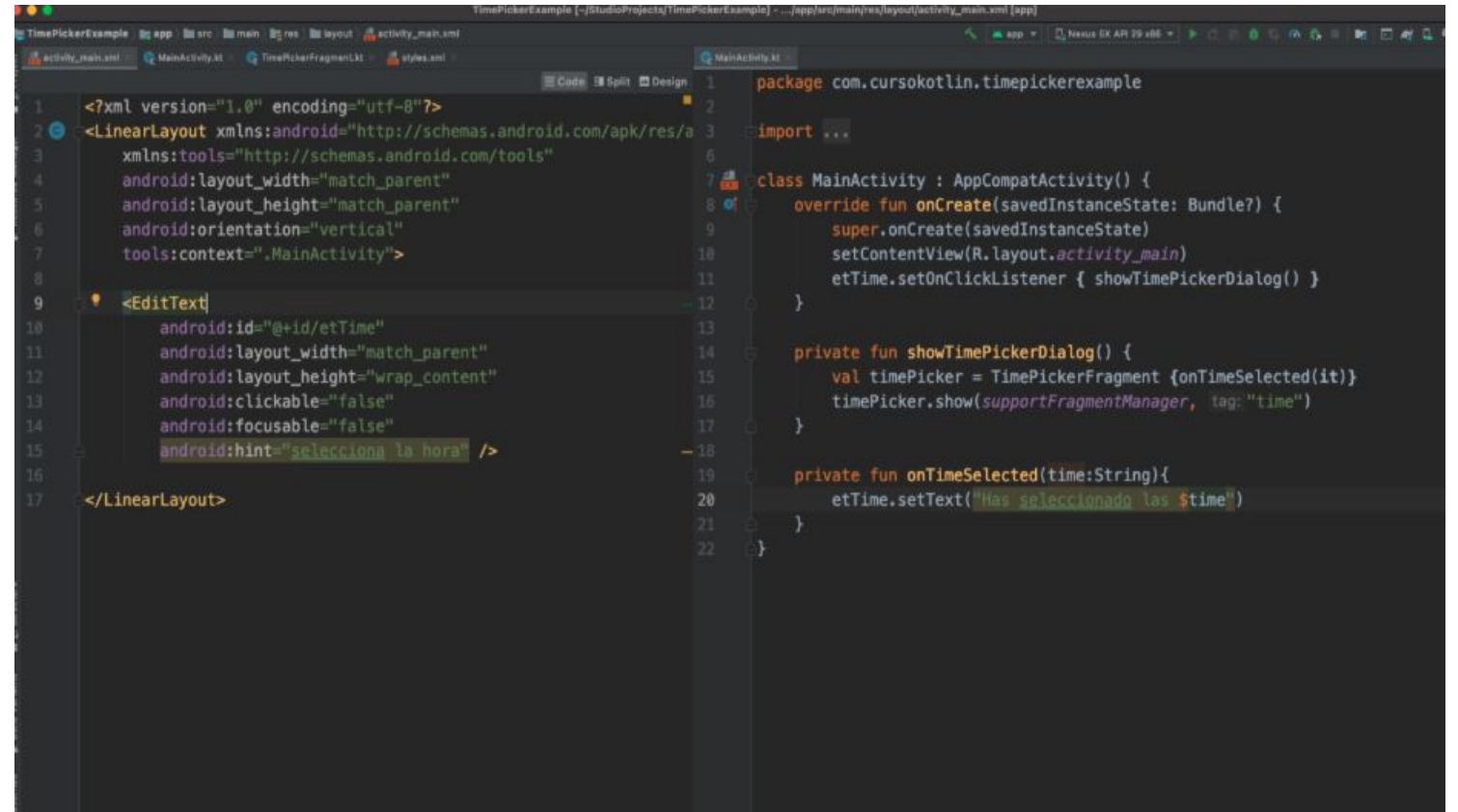
Hoy en día Kotlin es más adoptado como lenguaje por sus funcionalidades

```
/**
 * @author Pablo Ruiz Soria
 */
public class Lanzador {
    public static void main(String[] args) {
        float numero1 = 4;
        float numero2 = 7;
        float media1 = (numero1 + numero2) / 2;
        System.out.println("media1 = " + media1);
        float numero3 = 9;
        float numero4 = 6;
        float media2 = (numero3 + numero4) / 2;
        System.out.println("media2 = " + media2);
    }
}
```

Kotlin

¿Qué es?:

En 2017, Google anunció oficialmente que Kotlin se convirtió en un lenguaje de programación oficialmente compatible para el desarrollo de aplicaciones Android. Kotlin es un lenguaje moderno, conciso y seguro que ha ganado mucha popularidad en la comunidad de desarrolladores de Android debido a su legibilidad y facilidad de uso. En futuros módulos vamos a explorar mucho más Kotlin



```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/a
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">
    <EditText
        android:id="@+id/etTime"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:clickable="false"
        android:focusable="false"
        android:hint="selecciona la hora" />
</LinearLayout>
```

```
package com.cursokotlin.timepickerexample

import ...

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        etTime.setOnClickListener { showTimePickerDialog() }
    }

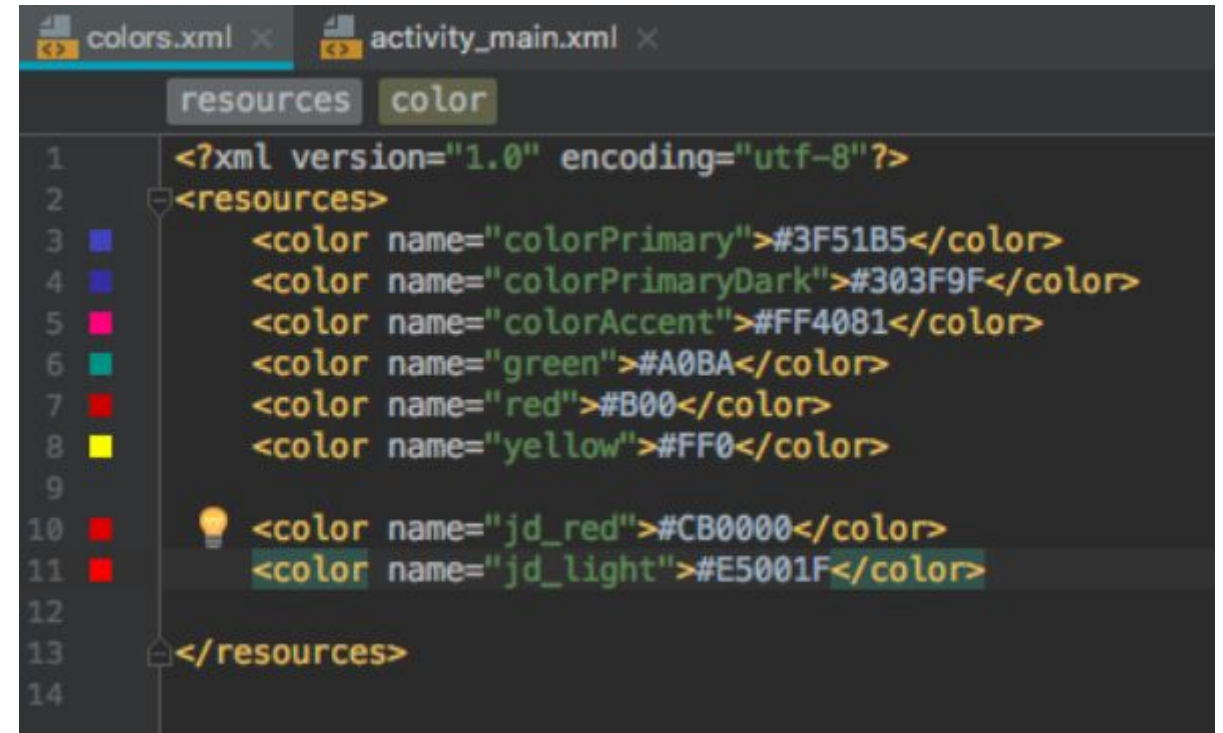
    private fun showTimePickerDialog() {
        val timePicker = TimePickerFragment {onTimeSelected(it)}
        timePicker.show(supportFragmentManager, tag: "time")
    }

    private fun onTimeSelected(time:String){
        etTime.setText("Has seleccionado las $time")
    }
}
```

XML

¿Qué es?:

Aunque no es un lenguaje de programación en sí, XML se utiliza ampliamente en Android para definir el diseño de la interfaz de usuario y la estructura de los recursos. Los archivos XML se utilizan para diseñar pantallas, definir recursos de diseño y declarar las vistas de la interfaz de usuario.



```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="colorPrimary">#3F51B5</color>
    <color name="colorPrimaryDark">#303F9F</color>
    <color name="colorAccent">#FF4081</color>
    <color name="green">#A0BA</color>
    <color name="red">#B00</color>
    <color name="yellow">#FF0</color>
    <color name="jd_red">#CB0000</color>
    <color name="jd_light">#E5001F</color>
</resources>
```

C/C++

¿Qué es?:

Si necesitas desarrollar componentes nativos de alto rendimiento, como bibliotecas de procesamiento de imágenes o videojuegos 3D, puedes utilizar C o C++ y luego integrar estas bibliotecas con tu aplicación Android.

```
input);  
input) >> dblTemp;  
t.length();  
) {  
ue;  
out[iLength - 3] != '.') {  
ue;  
< iLength) {  
t(sInput[iN])) {  
ue;  
(iN == (iLength - 3) ) {  
ue;
```


SQL

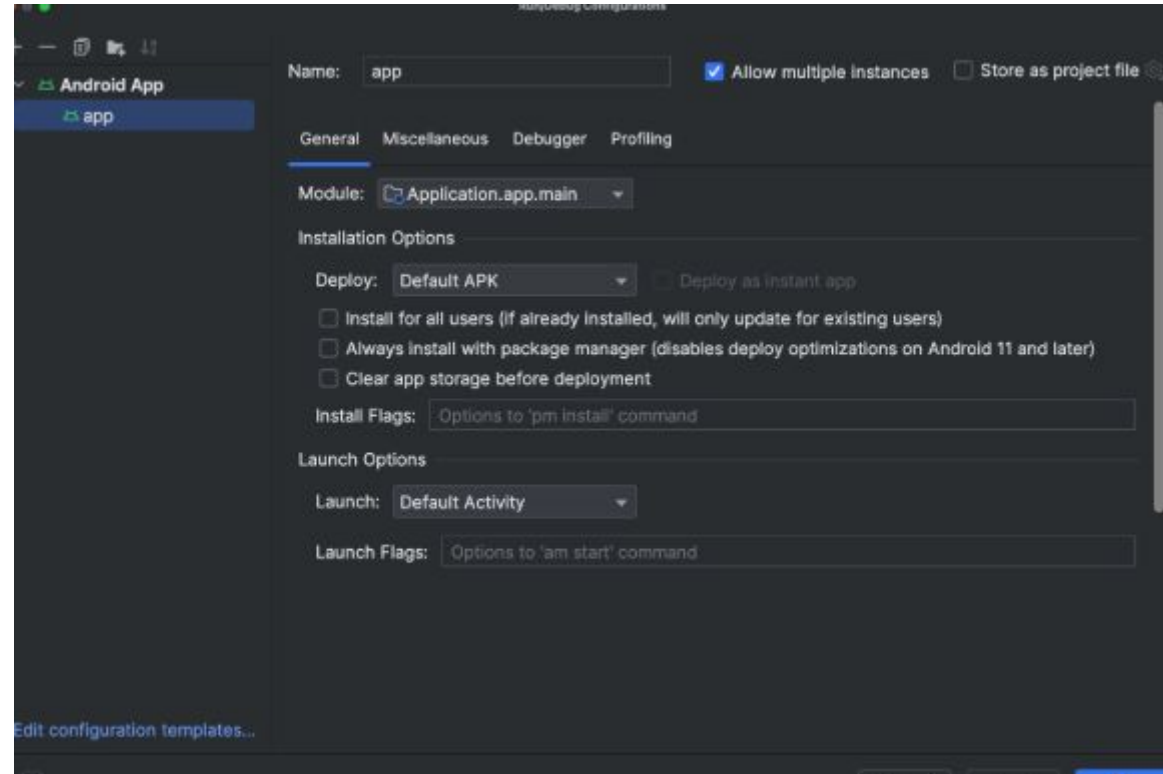
¿Qué es?:

Para interactuar con bases de datos en aplicaciones Android, a menudo se utiliza SQL para crear y administrar bases de datos SQLite incorporadas.

```
input);  
input) >> dblTemp;  
t.length();  
) {  
ue;  
out[iLength - 3] != '.') {  
ue;  
< iLength) {  
t(sInput[iN])) {  
ue;  
(iN == (iLength - 3) ) {  
ue;
```


Seleccionar lenguajes

La elección del lenguaje dependerá en gran medida de tus preferencias personales, las necesidades del proyecto y las herramientas y frameworks que estés utilizando. Java y Kotlin son los lenguajes más comunes y recomendados para el desarrollo de aplicaciones Android en la actualidad, y la elección entre ellos depende de factores como la familiaridad, la eficiencia y las características del lenguaje.





Momento:

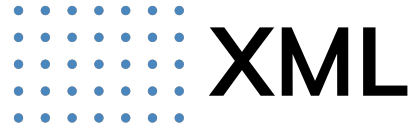
Time-out!



5 -10 min.



XML



¿Qué es?:

XML, Extensible Markup Language, es un lenguaje de marcado utilizado para almacenar y transportar datos de una manera legible tanto para humanos como para máquinas. Cuando hablamos de lenguaje de marcado, nos referimos a etiquetas. Si alguna vez viste o creaste el código de una página web, verás que su escritura es muy similar

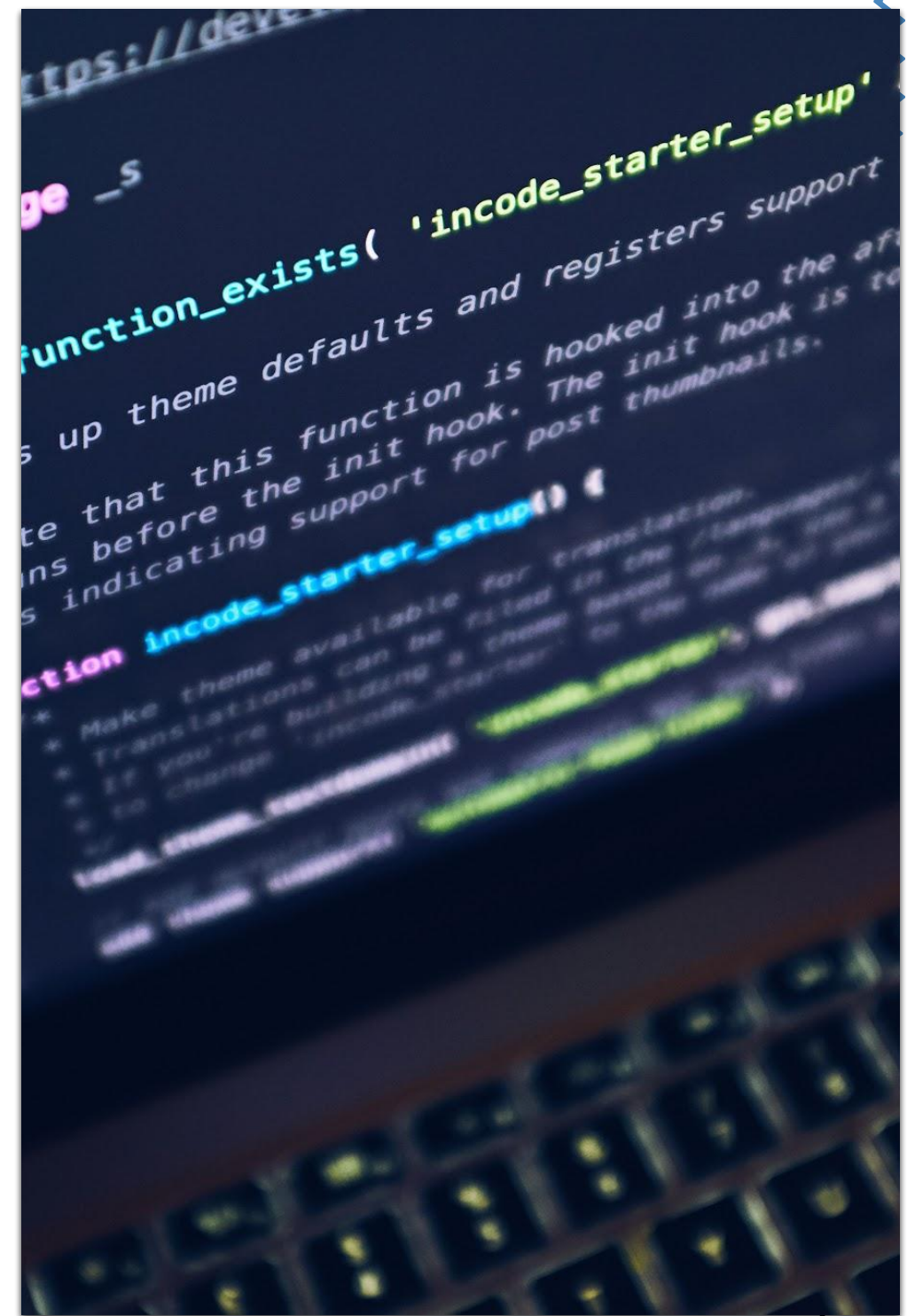
Pero en nuestro caso, lo utilizaremos, no sólo para describir la estructura, sino también el contenido de los archivos de recursos, incluidos los archivos de diseño de interfaz de usuario.

XML

¿En android?:

En Android, XML se utiliza para definir la estructura y las propiedades de los componentes de la interfaz de usuario, como botones, campos de texto, imágenes, y otros elementos más.

También podemos pensarlo como un medio para describir cómo se verá la interfaz de usuario de una aplicación. En lugar de escribir código de programación directamente, los desarrolladores pueden utilizar XML para organizar y definir los componentes visuales de la aplicación de manera más intuitiva.

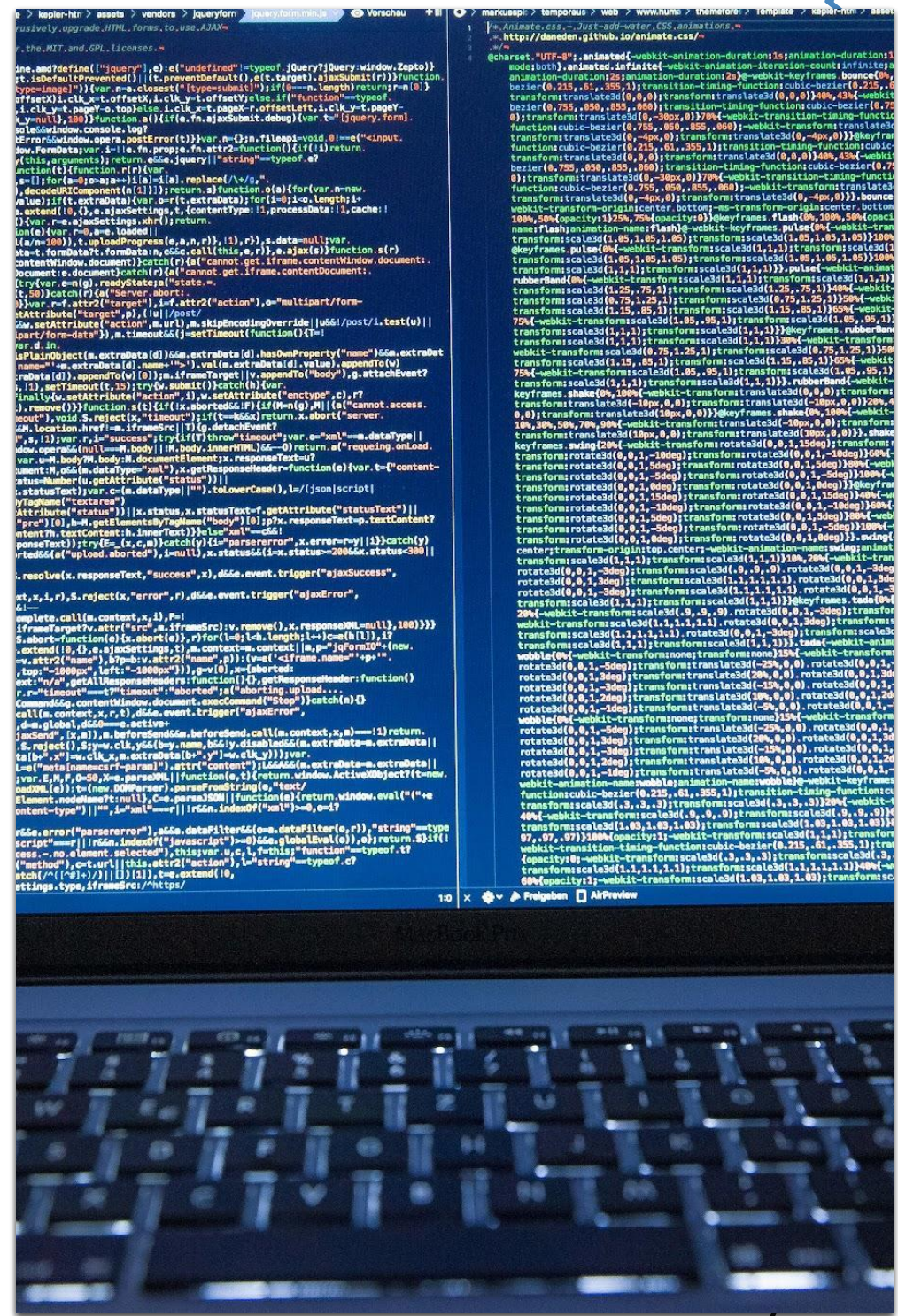


Estructura de un documento XML

¿En android?:

En Android, XML se utiliza para definir la estructura y las propiedades de los componentes de la interfaz de usuario, como botones, campos de texto, imágenes, y otros elementos más.

También podemos pensarlo como un medio para describir cómo se verá la interfaz de usuario de una aplicación. En lugar de escribir código de programación directamente, los desarrolladores pueden utilizar XML para organizar y definir los componentes visuales de la aplicación de manera más intuitiva.



■ Estructura de un documento XML

¿Cuáles son?:

Elementos: Son unidades lógicas que representan datos y se construyen mediante etiquetas de inicio y fin que encierran una cadena de texto o contenido. Pueden existir elementos vacíos, indicados por etiquetas que se cierran a sí mismas, como `<br/>`, y algunos elementos pueden tener atributos que definen propiedades adicionales del elemento.

Instrucciones: Son órdenes especiales para ser utilizadas por la aplicación que procesa.

`<?xml-stylesheet type="text/css" href="estilo.css">`

Las instrucciones XML comienzan por “<?” y terminan “por ?>”.

Comentarios: Consisten en la información que no forma parte del documento.

Comienzan por “<!--” y terminan por “-->”.



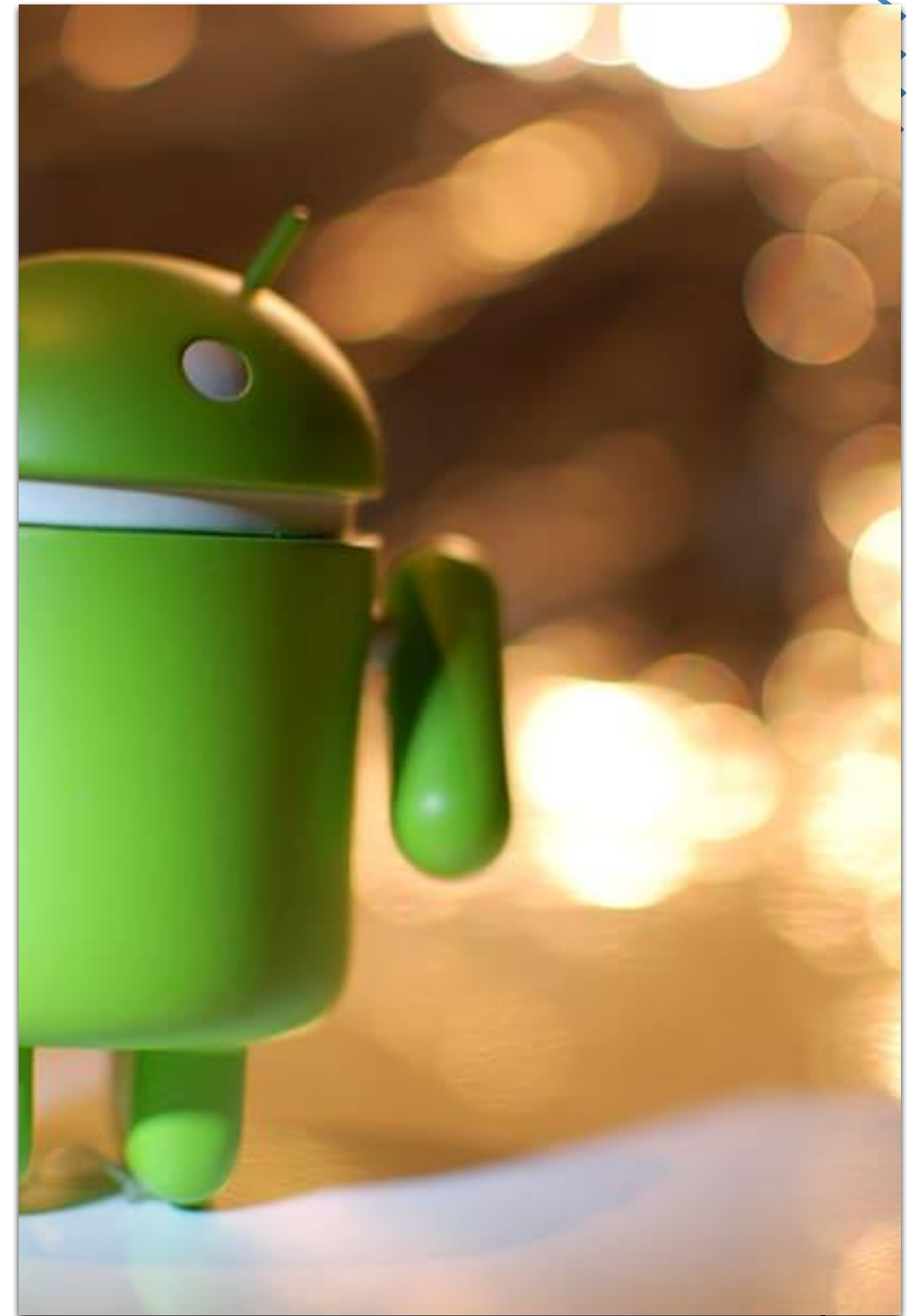
Sintaxis XML

¿Cuáles es?:

La declaración XML es sensible a mayúsculas y minúsculas y debe empezar por "<?xml>" donde "xml" se escriben en minúsculas.

Si el documento contiene declaración XML, estrictamente tiene que ser la primera declaración del documento XML.

La declaración XML debe ser estrictamente la primera declaración en el documento XML.



Sintaxis XML

Elementos:

Sintaxis de Elemento: XML Cada elemento tiene que estar cerrada o con inicio o final con elementos como se muestra a continuación:

```
<element>....</element>
```

O en simple de los casos, sólo en esta forma:

```
<element/>
```



Sintaxis XML

Anidamiento de elementos:

un elemento XML puede contener varios XML de elementos como el de los niños, pero los niños no deben superponerse elementos, es decir, una etiqueta de cierre del elemento debe tener el mismo nombre que el de la más reciente etiqueta de inicio sin precedentes.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <!-- Un EditText para ingresar texto -->
    <EditText
        android:id="@+id/editText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Escribe algo aquí" />

    <!-- Un Button para enviar el texto -->
    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Enviar" />

    <!-- Un TextView para mostrar texto -->
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Texto aparecerá aquí"
        android:textSize="18sp" />

</LinearLayout>
```

LIVE CODING



Ejemplo XML:

Crear una breve demostración de un archivo XML donde se muestren todas sus secciones y formato

LIVE CODING

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp"
    android:gravity="center">
    <EditText
        android:id="@+id/editTextUsername"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Nombre de usuario"
        android:inputType="text" />

    <EditText
        android:id="@+id/editTextPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Contraseña"
        android:inputType="textPassword" />

    <Button
        android:id="@+id/buttonLogin"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Iniciar sesión" />
</LinearLayout>
```

Layout



Layouts



¿Qué son?:

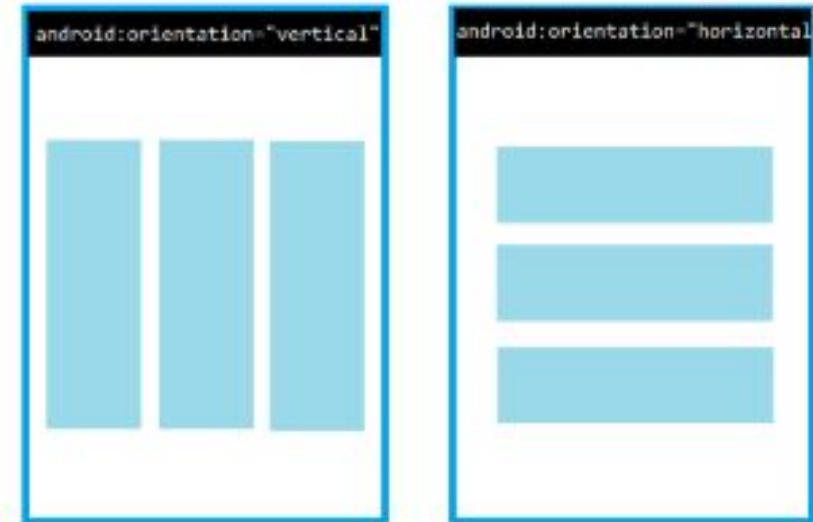
Vamos a definirlos como la estructura visual que nos permite el posicionamiento de los componentes, en cuanto a lo que corresponde a la interfaz de usuario. Es decir, que es donde vamos a estar acomodando todos los elementos visual que ve nuestro usuario final. Estos pueden ser botones, imágenes, campos de texto, u otros elementos.

LinearLayout

¿Qué es?:

Este layout nos permite colocar los elementos en una línea horizontal o vertical. Nos permite especificar pesos para controlar cómo se distribuye el espacio disponible entre los componentes.

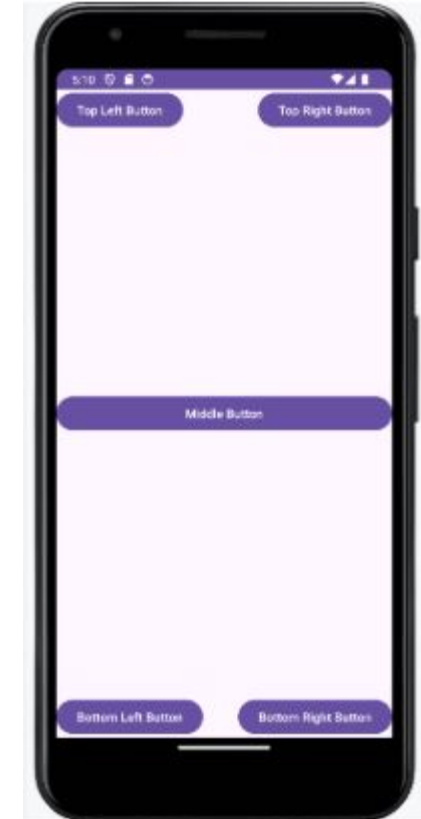
Es el más simple de los layouts y es muy usado cuando solo queremos establecer un orden con los elementos.



RelativeLayout

¿Qué es?:

Nos permite alinear los elementos en relación con otros elementos o con los bordes de la pantalla. Se utiliza para crear diseños más complejos y flexibles.

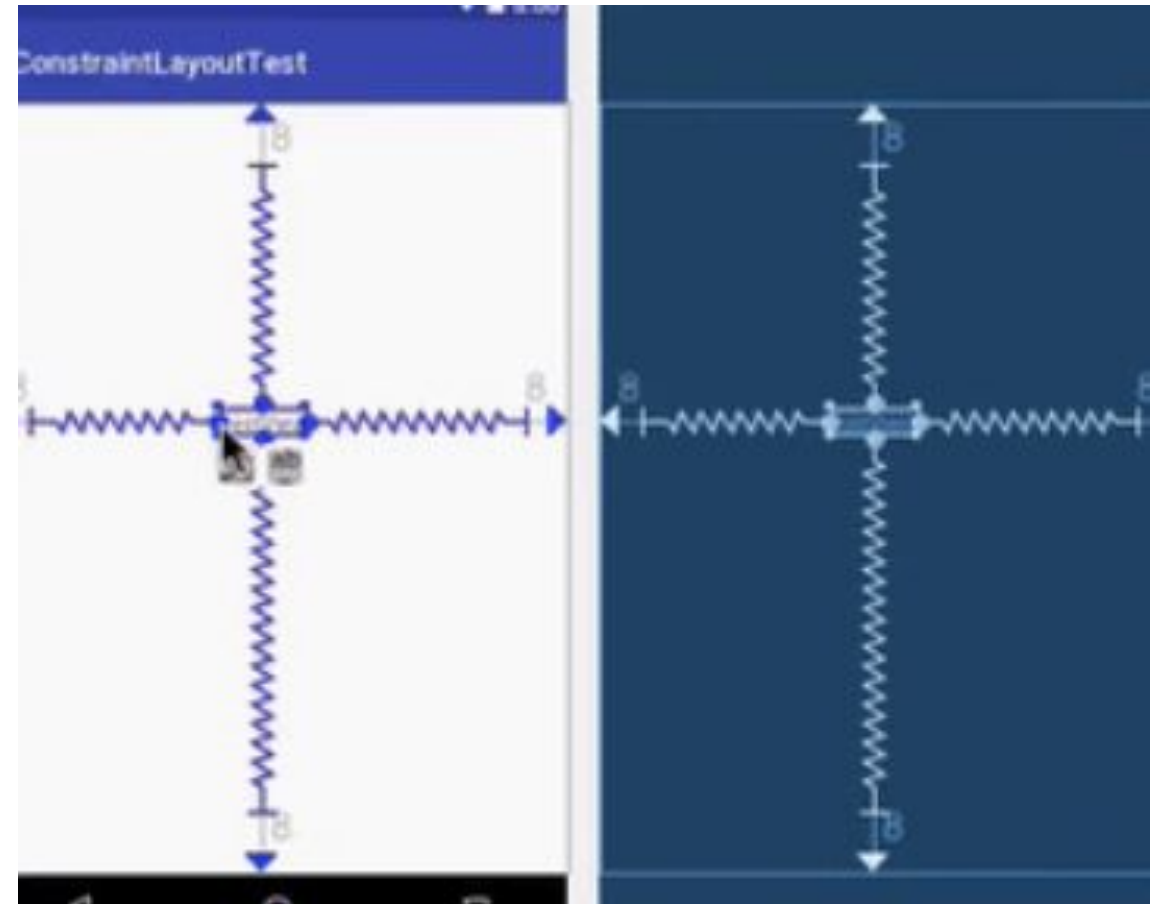


ConstraintLayout

¿Qué es?:

En este caso, se introducen restricciones para definir las relaciones entre los componentes y su posición relativa. Lo cual proporciona una forma más eficiente y flexible de diseñar interfaces de usuario complejas.

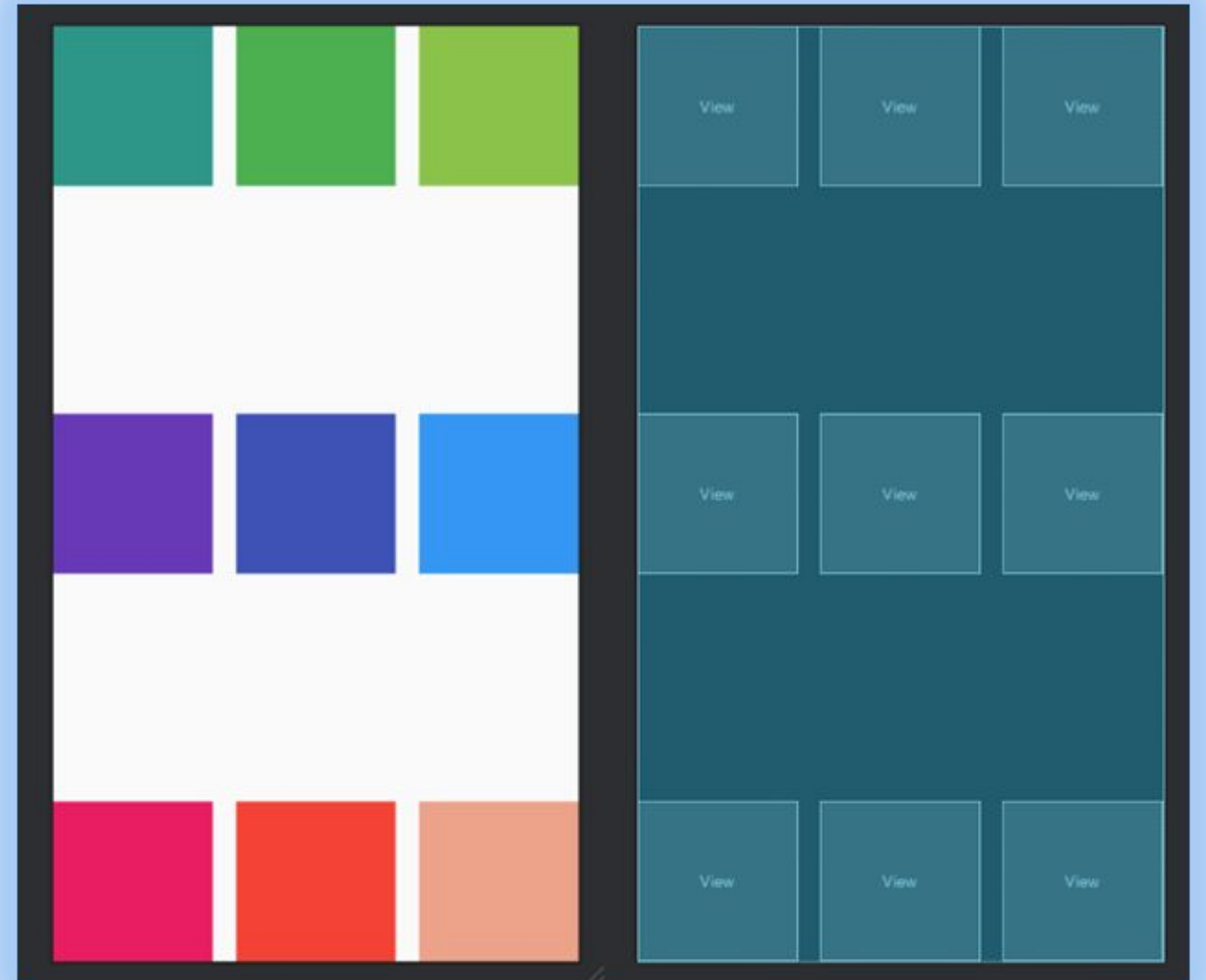
Uno de los Layouts más usados para pantallas más complejas.



FrameLayout

¿Qué es?:

Permite colocar los elementos uno encima del otro, lo cual permite que se superpongan los elementos en la pantalla. Es útil para crear capas y efectos de superposición.

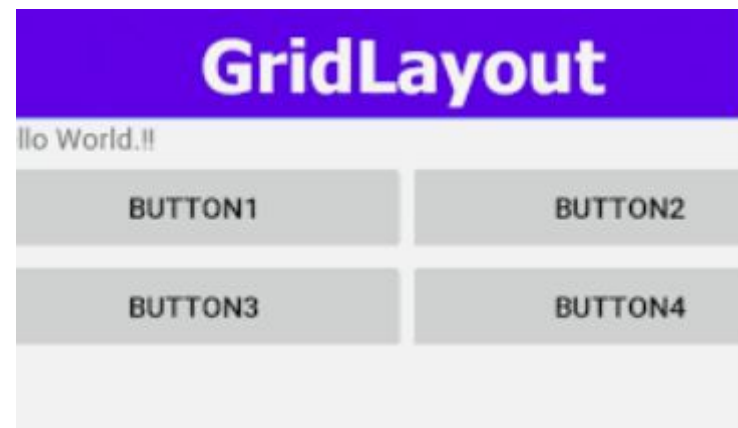




GridLayout

¿Qué es?:

Nos permite organizar los componentes en una cuadrícula de filas y columnas. Lo utilizamos mayormente para crear diseños estructurados y ordenados.



¿Como elegir?

Requisitos de diseño: Es bueno tener definido cómo deseas que se vea y se comporte la interfaz de usuario de tu aplicación. Debes evaluar los requisitos de diseño para determinar qué tipo de layout se podrá ajustar mejor. Es recomendable hacerte algunas preguntas como por ejemplo ¿Necesito que los elementos se ajusten de manera horizontal o vertical?

Estructura y jerarquía: Es muy conveniente analizar la estructura y la jerarquía de los elementos. Hay que considerar cómo se relacionan entre sí los componentes y qué layout puede facilitar su posicionamiento y disposición.



¿Como elegir?

Flexibilidad y adaptabilidad: Debemos pensar en la flexibilidad y adaptabilidad que queremos darle a la interfaz de usuario. Por ejemplo, podríamos necesitar que los elementos se ajusten automáticamente a diferentes tamaños de pantalla y orientaciones. Un ejemplo de layout podría ser `ConstraintLayout`.

Rendimiento: Es muy importante tener en cuenta el rendimiento de la aplicación y evaluar las necesidades de rendimiento de tu aplicación. Un ejemplo de layout podría ser `LinearLayout`, ya que puede ser más eficiente en términos de rendimiento y recursos.



LIVE CODING



Ejemplo de Layouts:

Crear un ejemplo donde podamos apreciar el uso de `LinearLayout` y `RelativeLayout` para mostrar sus diferencias y características.

¿Alguna consulta?





Momento:

Time-out!



5 -10 min.



Vistas Básicas



Vistas Básicas

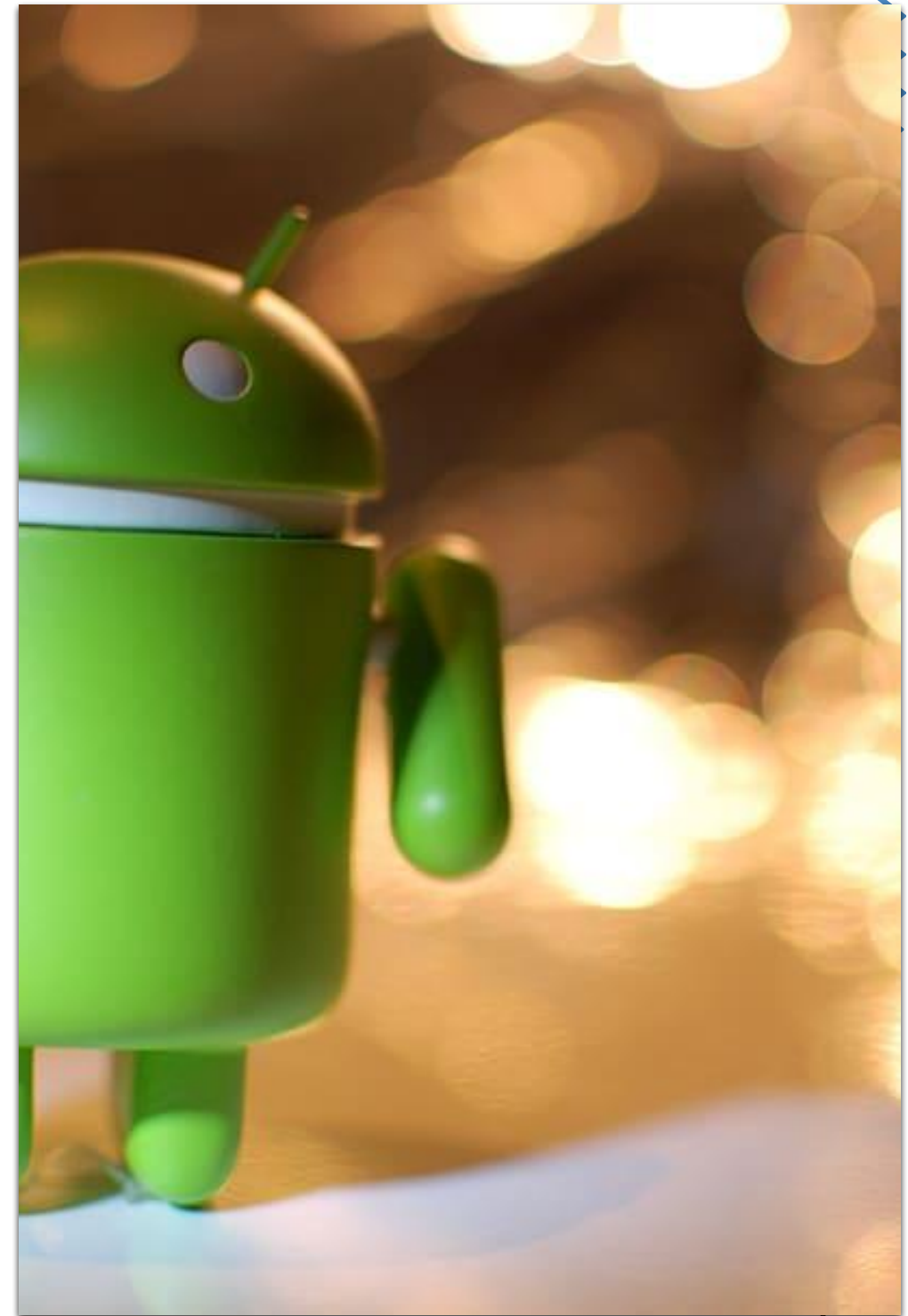
¿Qué son?:

Denominamos vistas Básicas a los componentes visuales más simples y comunes que usamos al crear interfaces gráficas en Android. La gran mayoría de las pantallas que vamos a ver en aplicaciones comerciales son una combinación de vistas básicas en diferentes configuraciones y orden.

TextView

TextView:

Es un componente que se utiliza para mostrar texto en la interfaz de usuario. Se usa para mostrar etiquetas, títulos, descripciones y cualquier otro tipo de texto.



EditText

EditText :

Es un componente que permite al usuario introducir texto. Se utiliza para obtener información del usuario, como nombres, contraseñas, etc.



Button

Button:

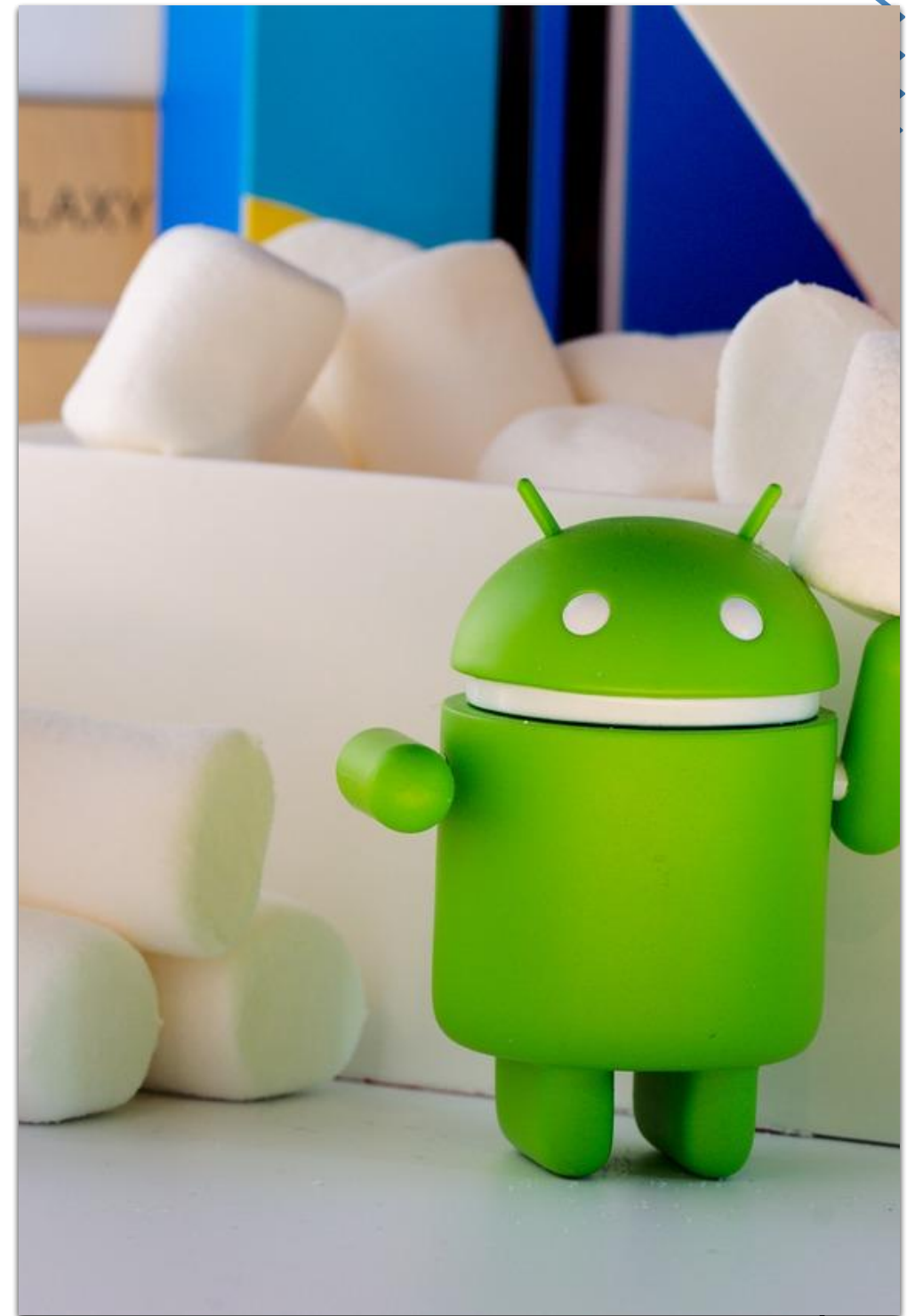
Es un componente que representa un botón en la interfaz de usuario. Se utiliza para iniciar acciones o procesos cuando se hace clic.



ImageView

ImageView:

Es un componente que se utiliza para mostrar imágenes.
Se utiliza para mostrar logotipos, fotos u otros gráficos.



CheckBox

CheckBox:

Es un componente que representa una casilla de verificación que puede estar marcada o desmarcada.
Se utiliza cuando se quiere permitir que el usuario seleccione varias opciones de una lista.

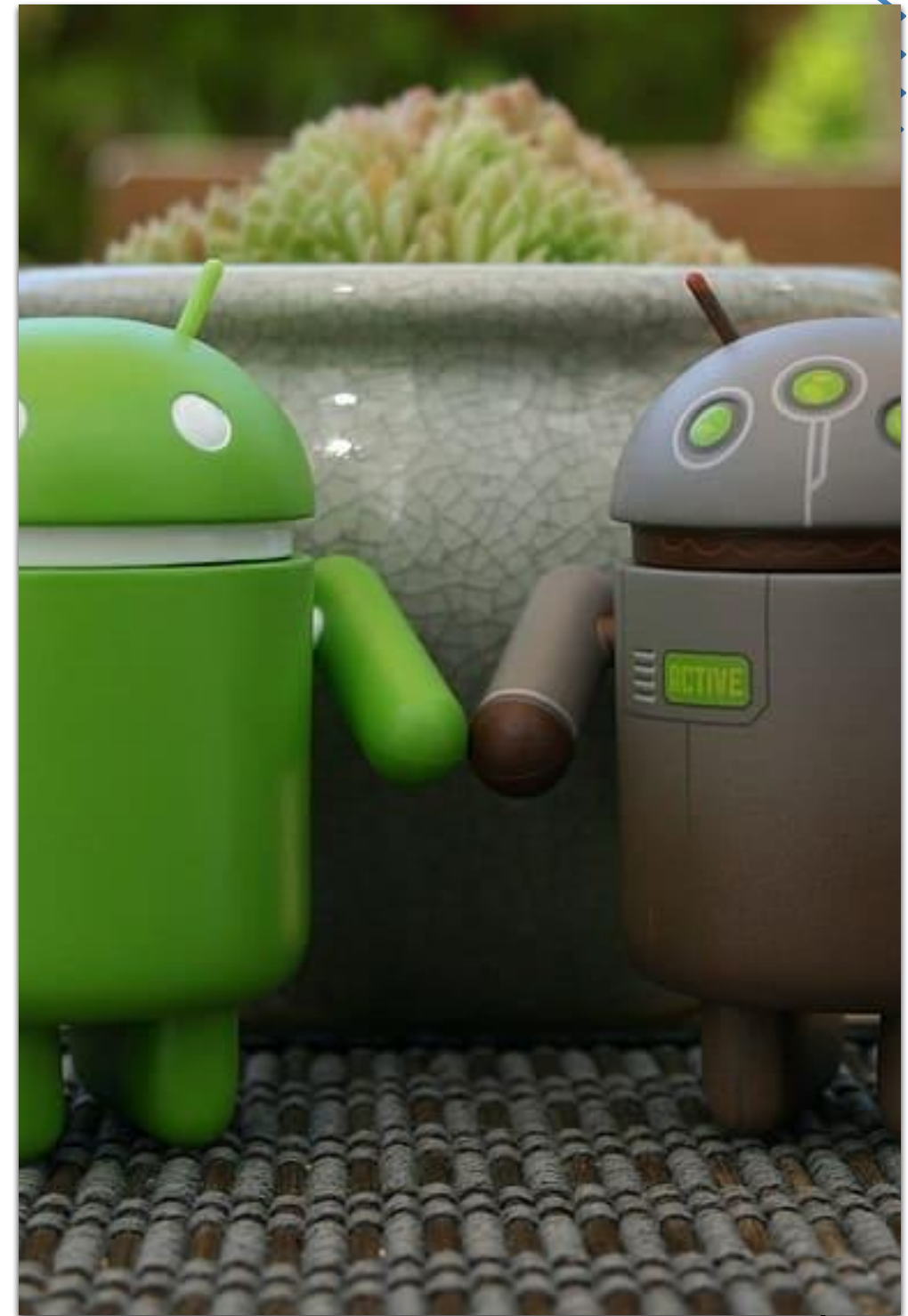


RadioButton

RadioButton:

Es un componente similar a un botón de opción que permite al usuario seleccionar una opción de un conjunto.

Se utiliza en grupos donde solo se puede seleccionar una opción a la vez.



ToggleButton

ToggleButton:

Es un botón de interruptor que tiene dos estados: activado y desactivado. Se utiliza cuando se quiere permitir al usuario alternar entre dos estados.



Controles de entrada

Entradas y Controles

A veces necesitas recibir algún tipo de input o entrada para controlar tus aplicaciones. Afortunadamente, Android provee de varias subclases View específicamente para este propósito.

Button

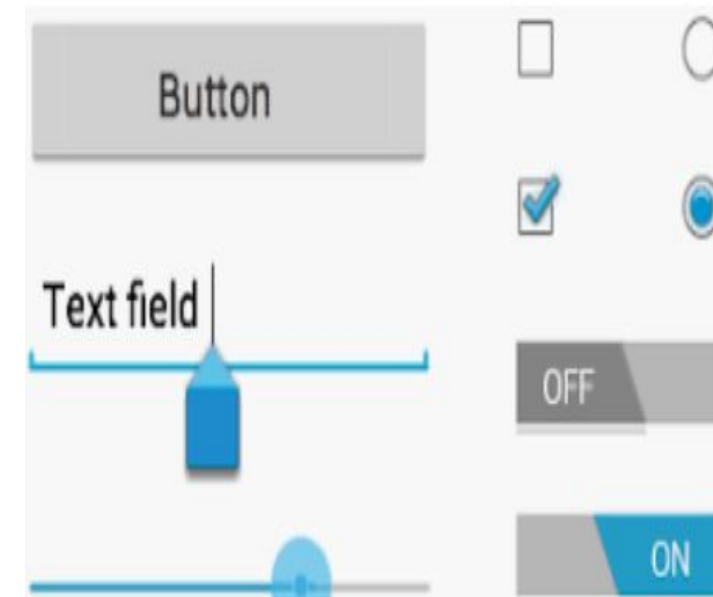
La clase Button es uno de los controles más básicos disponibles en la app. Escucha los clicks del usuario y llama a un método en tu código, de forma que puedas responder apropiadamente.

Switch y CheckBox

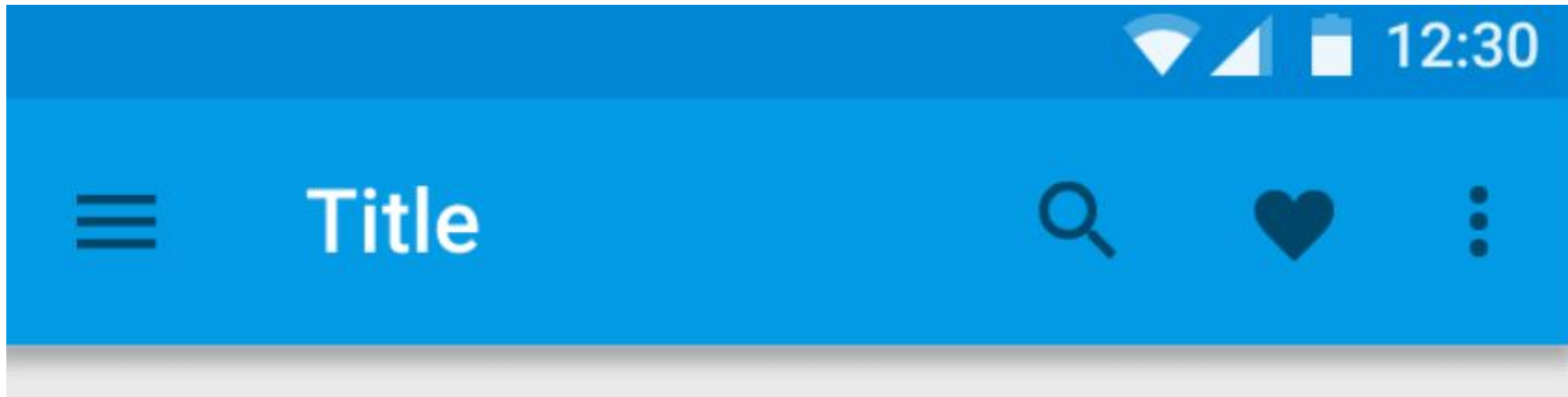
Las clases Switch y CheckBox tienen un estado activo e inactivo entre los que pueden alternarse. Esto es útil para cambiar los ajustes en una app. Versiones compatibles con Material Design, están disponibles a través de [AppCompat support library](#)

EditText

Esta subclase View es una extensión de la clase TextView y permite a los usuarios actualizar el texto que figura a través de una entrada de teclado.



Action Bar





Action Bar

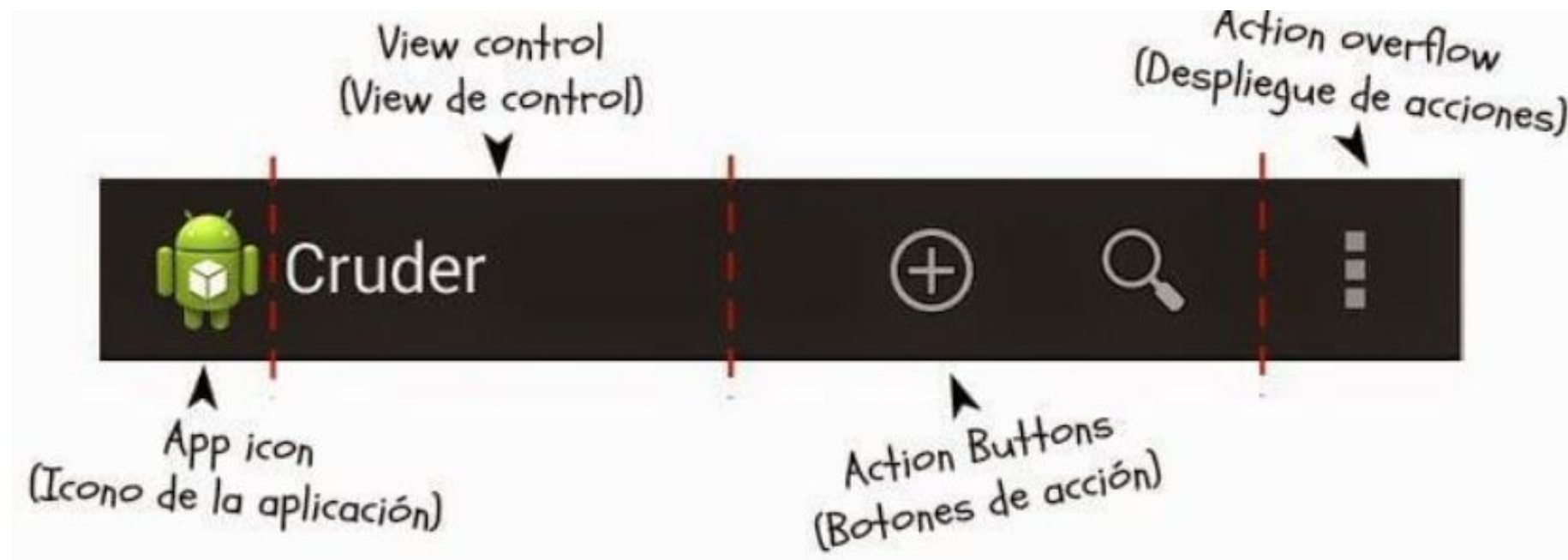
¿Qué es y cómo funciona?

Para quienes tienen dispositivos móviles ya puede deducir algunas funciones de este maravilloso elemento. La Action Bar es un **menú auxiliar** de las aplicaciones Android, que se ubica en la parte superior de cada actividad. Digo de cada actividad, ya que es un elemento que por lo general es persistente.

Esta pequeña barra tiene enormes utilidades, como por ejemplo: Proveernos acceso rápido a las acciones más comunes y solicitadas por los usuarios, organizar la navegación entre actividades (**pestañas para swiping, expand and collapse, navigation drawers**, etc.), proporcionarnos un espacio donde diferenciar nuestra aplicación de otras aplicaciones (a través del título, iconos particulares y demás) y proyectar fácil acceso de las funcionalidades de la aplicación.

Anatomía APP Bar

La barra de acción se divide en cuatro partes fundamentales que debemos reconocer antes de empezar a programar sobre ella. Por lo que veremos la siguiente ilustración sobre su estructura:



Anatomía APP Bar

Observemos la definición de cada segmento:

Icono de la aplicación: Como ya hemos visto, en esta ubicación se proyecta el recurso drawable del icono de la aplicación. Por defecto se ha usado un icono predeterminado por los recursos de Android, pero es ideal darle vida a tu aplicación diseñando tu propia imagen, logo o distinción.

View de control: Este espacio esta diseñado para insertar views que permitan acceder al contenido de la actividad con el fin de mejorar la navegación. Por lo general se usan **Spinners, TextViews, SearchViews**, etc., para controlar el contenido. Normalmente vemos un texto estático que visualiza el nombre de la aplicación.

Botones de acción: Representan las acciones más populares dentro de la aplicación, las cuales podemos ejecutar rápidamente al presionarlos.

Despliegue de acciones: Este segmento contiene una lista de acciones que no son tan populares, pero pueden ser necesitadas en algún momento por el usuario para tener acceso de forma sencilla.



Componentes del tipo selección

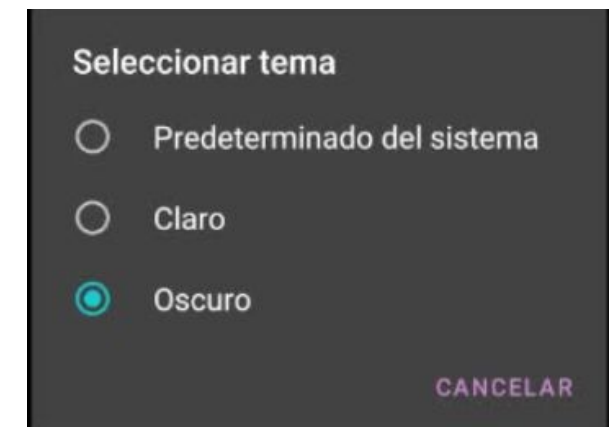
Selección Controls

Componente de tipo selección en Android

En primer lugar sería conveniente refrescar qué entendemos por un componente de tipo selección y por qué son tan útiles:

Un componente de selección, como su propio nombre indica, es un control que **nos permite seleccionar una opción entre una lista de posibilidades**, permitiendo así poder **interactuar** con el usuario, **mostrándole información** y realizando diferentes **acciones** cuando este **selecciona** un elemento u otro.

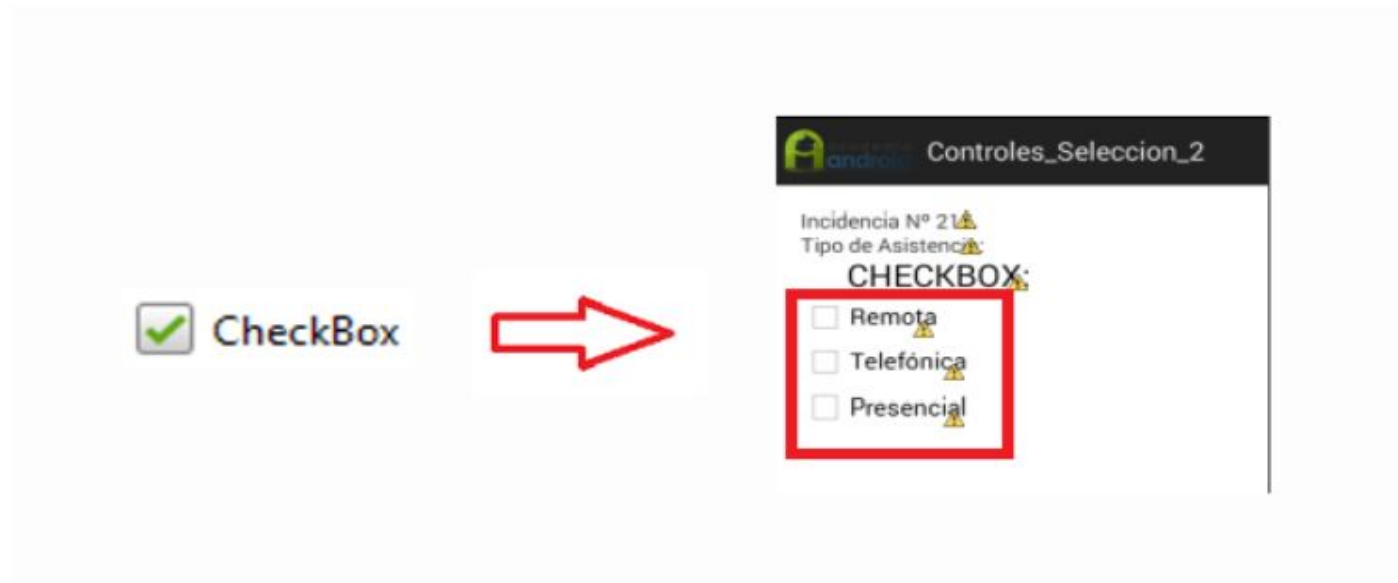
Además nos **ofrecen mayor seguridad** al utilizar **valores predefinidos**, de cara a posibles errores que puedan producir el uso indebido de ciertos widgets, como pueden ser los EditText. Si no se implementan de manera adecuada, pueden permitir introducir datos no válidos, y por lo tanto un mal funcionamiento de la aplicación.



Selección Controls Checkbox

CheckBox

Lo implementa la Clase CheckBox (Hereda de la Clase TextView), que proporciona dos estados (checked y unchecked), para poder marcar o desmarcar opciones al realizar click sobre el.



Selección Controls RadioButton

RadioButton

La Clase RadioButton (Hereda de la Clase CompoundButton y TextView) permite al usuario seleccionar una opción a partir de un conjunto. A diferencia de un CheckBox, este tipo de control se utiliza cuando deseamos que el resto de las opciones posibles sean excluidas de la selección. Este tipo de componente es muy útil para mostrar las posibilidades de selección de que dispone el usuario.



Selección Controls Spinner

Spinner

La Clase Spinner nos proporciona una manera rápida de seleccionar un valor de una lista desplegable. En su estado normal, mostrará el valor seleccionado. Al pulsar sobre el control mostrará una lista desplegable con todos los valores disponibles desde dónde el usuario podrá seleccionar el que estime oportuno. Similar al componente ComboBox de otros FrameWork de desarrollo.



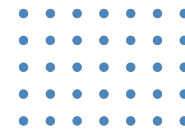
Selección Controls Gallery

Gallery

Componente que nos permite mostrar una lista de elementos, (normalmente para visualizar imágenes) que se desplaza de manera horizontal, permitiendo seleccionar uno de ellos, mostrando una apariencia muy intuitiva para el usuario. Esta Clase está desfasada para la API 16.



Listview

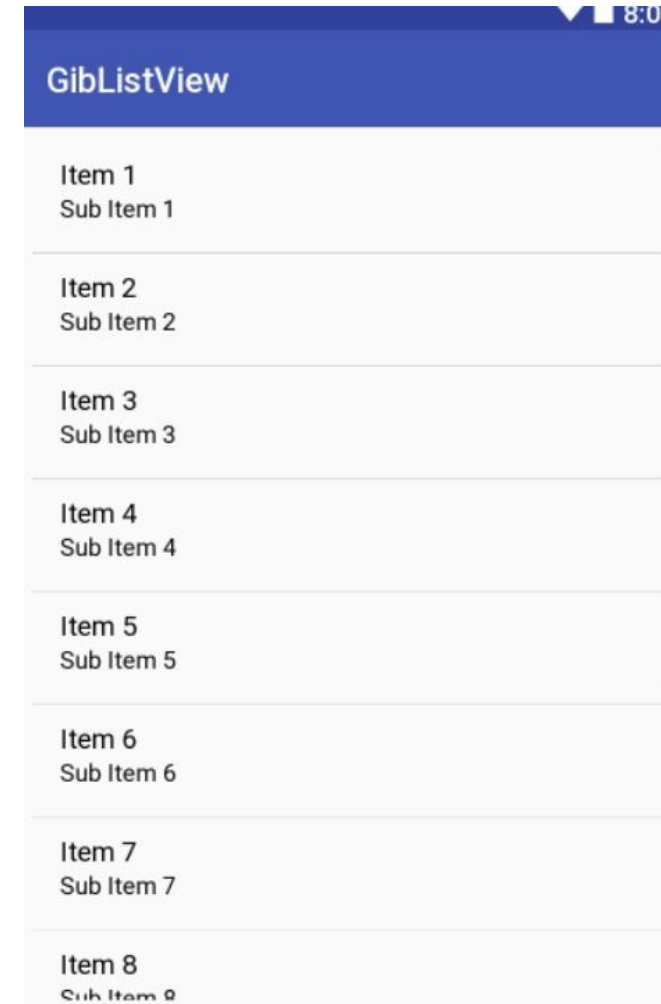


ListView

ListView que muestra una lista de datos estáticos

Paso 1. Crea un proyecto nuevo.

Paso 2. Coloca un ListView usando la vista Diseño.





ListView



Paso 3. Crea las variables a utilizar

Entre ellas, una variable de tipo *ListView* (en *MainActivity*)

```
private ListView listview;  
private ArrayList<String> names;
```

Paso 4. Enlazar

Enlaza la variable anterior con el elemento *ListView* de la vista Diseño.

```
listview = (ListView) findViewById(R.id.listview);
```

Paso 5. Coloca los datos que quieras mostrar en un Array de Strings

```
names = new ArrayList<String>();  
names.add("Veracruz");  
names.add("Tabasco");  
names.add("Chiapas");  
names.add("Campeche");  
names.add("Quintana Roo");
```

ListView

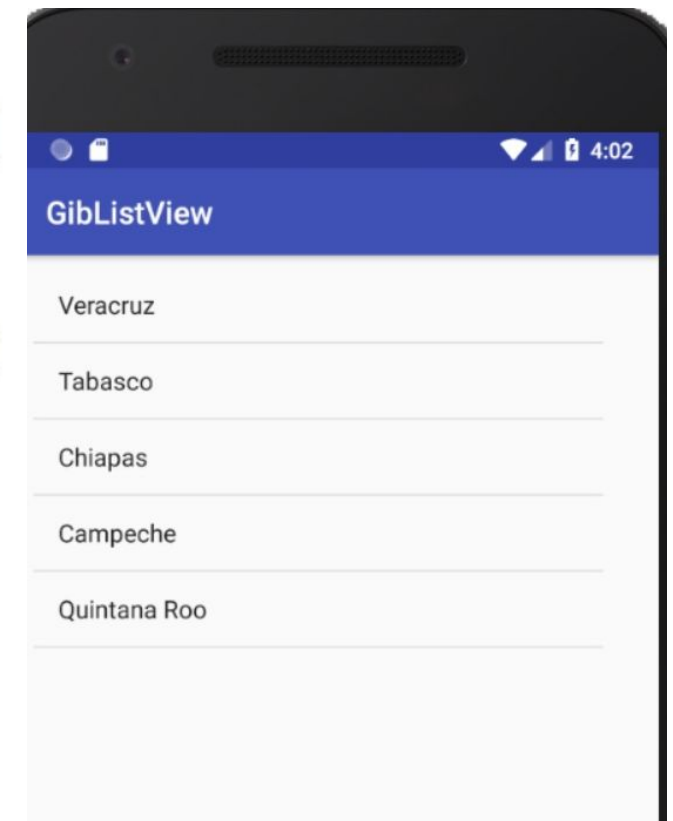
Paso 6. Crea un adaptador y agrégale la variable de Strings.

```
ArrayAdapter<String> adapter = new ArrayAdapter<String>(this, android.R.layout.simple_list_item_1, names);
```

Paso 7. Indica que el listview utilizará ese adaptador.

```
listview.setAdapter(adapter);
```

El resultado hasta aquí es el siguiente:



Cards

Cards

El widget [CardView](#) forma parte de las [Bibliotecas de compatibilidad v7](#). Para usarlo en tu proyecto, agrega la siguiente dependencia al archivo build.gradle del módulo de tu app:

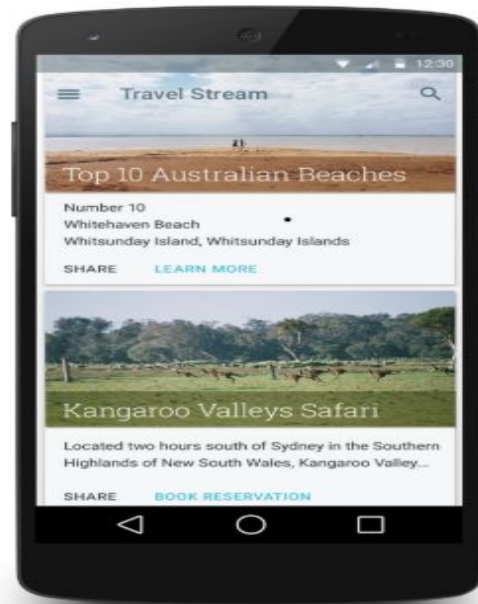


Figura 1: Ejemplos de tarjeta

```
dependencies {  
    implementation 'com.android.support:cardview-v7:28.0.0'  
}
```

Figura 1: Ejemplos de tarjeta



Cards

Cómo crear tarjetas

Para usar un objeto [CardView](#), necesitas agregarlo a tu archivo de diseño. Úsalo como grupo de vistas para incluir otras vistas. En este ejemplo, [CardView](#) incluye un objeto [TextView](#) individual que muestra parte de la información al usuario.

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    xmlns:card_view="http://schemas.android.com/apk/res-auto"
    ... >
    <!-- A CardView that contains a TextView -->
    <android.support.v7.widget.CardView
        xmlns:card_view="http://schemas.android.com/apk/res-auto"
        android:id="@+id/card_view"
        android:layout_gravity="center"
        android:layout_width="200dp"
        android:layout_height="200dp"
        card_view:cardCornerRadius="4dp">

        <TextView
            android:id="@+id/info_text"
            android:layout_width="match_parent"
            android:layout_height="match_parent" />
    </android.support.v7.widget.CardView>
</LinearLayout>
```



Cards



Las tarjetas se dibujan en la pantalla con una elevación predeterminada, y esto hace que el sistema dibuje una sombra debajo de ellas. Puedes ofrecer una elevación personalizada para una tarjeta con un atributo `card_view:cardElevation`. Si es así, se dibujará una sombra más pronunciada con una elevación más grande y una elevación más baja generará una sombra más suave. [CardView](#) usa la elevación real y sombras dinámicas de Android 5.0 (API nivel 21) y versiones posteriores, y regresa a la implementación de una sombra programática en las versiones anteriores.

Usa estas propiedades para personalizar la apariencia del widget [CardView](#):

- Para definir el radio de la esquina de tus diseños, usa el atributo `card_view:cardCornerRadius`.
 - Para definir el radio de la esquina de tu código, usa el método `CardView.setRadius`.
 - Para definir el color de fondo de una tarjeta, usa el atributo `card_view:cardBackgroundColor`.
- Para obtener más información, consulta la referencia de la API de [CardView](#).

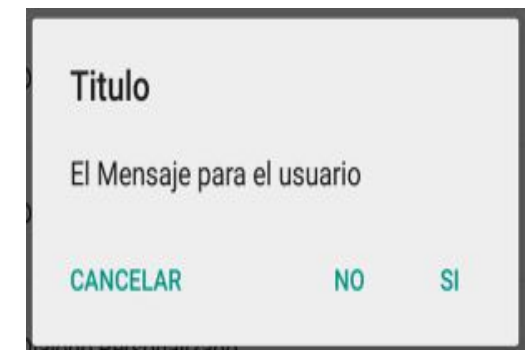
Diálogos en Android

Diálogos en Android

Un diálogo de alerta es aquel que está diseñado para mostrar un título, un mensaje o cuerpo y hasta 3 botones de confirmación en su zona de acción.

En los botones podremos ver acciones familiares como lo son la aceptación (Botón positivo) que determina que el usuario está de acuerdo con la acción. La cancelación (Botón negativo) para evitar la realización de una acción, y la neutralidad (Botón neutro) para determinar que aún no se está listo para proseguir o cancelar la acción.

Un diálogo de alerta se representa con la clase `AlertDialog`. En su construcción debes usar un elemento auxiliar llamado Builder, el cual te ayudará a definir las partes del diálogo con gran facilidad y sus eventos de respuesta para los botones.





Diálogos en Android

Builder te permitirá fabricar las características del diálogo a través de métodos `set*()`. Los más frecuentes son:

- **`setTitle()`**: Asigna una cadena al título del diálogo.
- **`setMessage()`**: Setea el mensaje que deseas transmitir en el contenido.
- **`setPositiveButton()`**: Crea una instancia del botón de confirmación. El primer parámetro que recibe es el texto del botón y el segundo una escucha `OnClickListener` para determinar qué acción se tomará al ser presionado.
- **`setNegativeButton()`**: Hace exactamente lo mismo que `setPositiveButton()` pero para el botón de decisión negativa.
- **`setNeutralButton()`**: Genera una escucha para el botón neutral. Al igual que los dos métodos anteriores, recibe el nombre y la escucha.

Diálogos en Android

Crear Dialogo Con Lista Simple

Si quieres crear una lista simple usa el método `setItems()` en vez de `setMessage()`. Este método recibe un arreglo de cadenas que permita definir las etiquetas de los elementos y una escucha `OnClickListener` para decidir que sucede...

```
/**
 * Crea un Diálogo con una lista de selección simple
 *
 * @return La instancia del diálogo
 */
public AlertDialog createSingleListDialog() {
    AlertDialog.Builder builder = new AlertDialog.Builder(getActivity());

    final CharSequence[] items = new CharSequence[3];

    items[0] = "Naranja";
    items[1] = "Mango";
    items[2] = "Banano";

    builder.setTitle("Diálogo De Lista")
        .setItems(items, new DialogInterface.OnClickListener() {
            @Override
            public void onClick(DialogInterface dialog, int which) {
                Toast.makeText(
                    getActivity(),
                    "Seleccionaste:" + items[which],
                    Toast.LENGTH_SHORT
                ).show();
            }
        });

    return builder.create();
}
```



Diálogos en Android

Crear Dialogo Con Lista De Radio Buttons

Ahora creemos una lista con **radio buttons**. Esta vez usaremos el método `setSingleChoiceItems()`, el cual también recibe un array de caracteres, el índice del radio seleccionado y una escucha para el manejo de los eventos:

```
/**
 * Crea un diálogo con una lista de radios
 *
 * @return Diálogo
 */
public AlertDialog createRadioListDialog() {
    AlertDialog.Builder builder = new AlertDialog.Builder(getActivity());

    final CharSequence[] items = new CharSequence[3];

    items[0] = "Soltero/a";
    items[1] = "Casado/a";
    items[2] = "Divorciado/a";

    builder.setTitle("Estado Civil")
        .setSingleChoiceItems(items, 0, new DialogInterface.OnClickListener() {
            @Override
            public void onClick(DialogInterface dialog, int which) {
                Toast.makeText(
                    getActivity(),
                    "Seleccionaste: " + items[which],
                    Toast.LENGTH_SHORT
                ).show();
            }
        });

    return builder.create();
}
```



Estado Civil

Soltero/a ☐

Casado/a ☒

Divorciado/a ☐

Diálogos en Android

Crear Dialogo Con Lista De Múltiple Selección De Checkboxes

La implementación de un diálogo con checklists requiere el uso del método `setMultipleChoiceItems()`. Al igual que los otros tipos de lista, este recibe un array con las etiquetas de cada elemento y una escucha para los eventos. Adicionalmente tiene un parámetro para indicar que checkboxes están seleccionados.

```
/**
 * Crea un diálogo con una lista de checkboxes
 * de selección múltiple
 *
 * @return Diálogo
 */
public AlertDialog createMultipleListDialog() {
    AlertDialog.Builder builder = new AlertDialog.Builder(getActivity());

    final ArrayList itemsSeleccionados = new ArrayList();

    CharSequence[] items = new CharSequence[3];

    items[0] = "Desarrollo Android";
    items[1] = "Diseño De Bases De Datos";
    items[2] = "Pruebas Unitarias";

    builder.setTitle("Intereses")
        .setMultiChoiceItems(items, null, new DialogInterface.OnMultiChoiceClickListener() {
            @Override
            public void onClick(DialogInterface dialog, int which, boolean isChecked) {
                if (isChecked) {
                    // Guardar índice seleccionado
                    itemsSeleccionados.add(which);
                    Toast.makeText(
                        getActivity(),
                        "Checks seleccionados:(" + itemsSeleccionados.size() + ")",
                        Toast.LENGTH_SHORT)
                        .show();
                } else if (itemsSeleccionados.contains(which)) {
                    // Remover índice sin selección
                    itemsSeleccionados.remove(Integer.valueOf(which));
                }
            }
        });

    return builder.create();
}
```

Intereses

Desarrollo Android	☒
Diseño De Bases De Datos	☒
Pruebas Unitarias	☐

LIVE CODING



Ejemplos de vistas:

Mostrar breves ejemplos de cada una de las vistas en XML y como se ven en Android Studio y en el dispositivo Android

Editor de vistas

Editor de vistas

¿Qué es?:

Android Studio proporciona un Editor de Vistas que es una herramienta gráfica para diseñar la interfaz de usuario de una aplicación Android. Este editor, conocido como "Layout Editor", permite a los desarrolladores crear y modificar las interfaces de usuario de sus aplicaciones de manera visual, sin necesidad de editar manualmente el código XML.



Características

Arrastrar y Soltar: Los desarrolladores pueden arrastrar y soltar componentes gráficos, como botones, texto, imágenes, etc., directamente en la interfaz de usuario desde la paleta de componentes.

Visualización en Tiempo Real: Los cambios realizados en el Editor de Vistas se reflejan en tiempo real, lo que facilita la visualización y ajuste de la interfaz de usuario mientras se trabaja.

Vista Preliminar de Temas y Dispositivos: Permite previsualizar cómo se verá la interfaz de usuario en diferentes versiones de Android y en diferentes tamaños de pantalla.

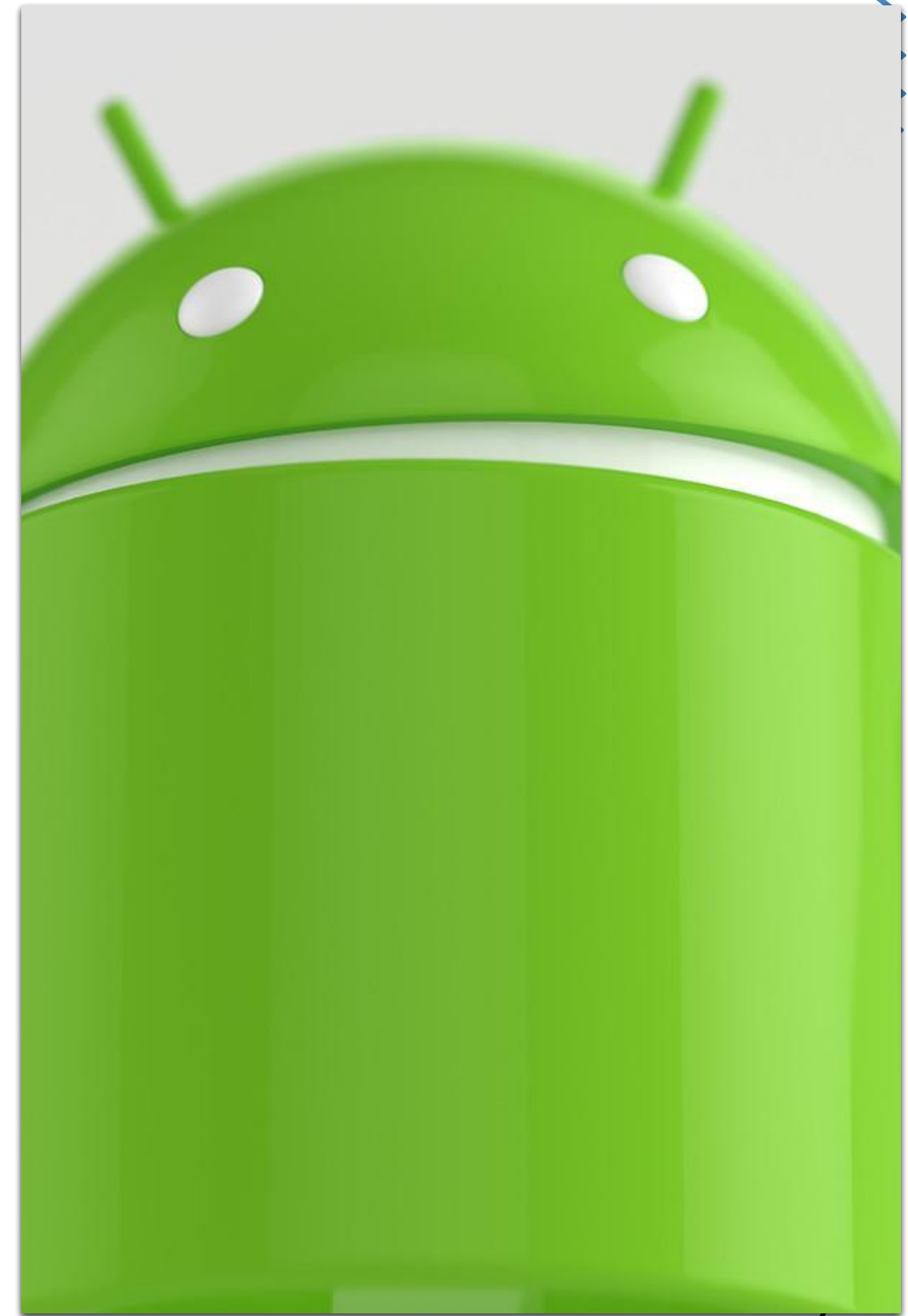


Características

Edición de Atributos: Los desarrolladores pueden modificar los atributos de los componentes directamente en el editor, lo que se refleja automáticamente en el código XML subyacente.

Vista de Árbol de Jerarquía: Proporciona una vista jerárquica de la estructura de la interfaz de usuario, lo que facilita la navegación y la comprensión de la disposición de los elementos.

Vista de Código XML: Aunque el Editor de Vistas es visual, los desarrolladores pueden cambiar fácilmente al modo de vista de código XML para realizar ajustes más detallados o para trabajar directamente en el código si lo prefieren.



LIVE CODING

Ejercicio: Formulario de Registro Simple

Objetivo:

Crear una pantalla de formulario usando **XML** para diseñar la interfaz gráfica. Al presionar un botón, se mostrará la información ingresada en un **Toast**.

Duración: 20 minutos

Enfoque: 80% diseño (XML), 20% lógica mínima en Java

Resultado esperado:

Formulario con los siguientes campos:

- Nombre (EditText)
- Correo electrónico (EditText)
- Contraseña (EditText, tipo password)
- Botón "Registrar"
- Al presionar el botón, mostrar los datos ingresados con **Toast**.

Tiempo: 20 Minutos

LIVE CODING

📁 Estructura del proyecto

- **1. activity_main.xml**
- xml
- CopiarEditar
- ```
<?xml version="1.0" encoding="utf-8"?>
```
- ```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
```
- ```
 android:layout_width="match_parent"
```
- ```
    android:layout_height="match_parent"
```
- ```
 android:padding="24dp"
```
- ```
    android:orientation="vertical"
```
- ```
 android:gravity="center">
```
- 
- ```
    <EditText
```
- ```
 android:id="@+id/editTextNombre"
```
- ```
        android:layout_width="match_parent"
```
- ```
 android:layout_height="wrap_content"
```
- ```
        android:hint="Nombre" />
```
-
- ```
 <EditText
```
- ```
        android:id="@+id/editTextEmail"
```
- ```
 android:layout_width="match_parent"
```
- ```
        android:layout_height="wrap_content"
```
- ```
 android:hint="Correo Electrónico"
```
- ```
        android:inputType="textEmailAddress"
```
- ```
 android:layout_marginTop="10dp" />
```
- 
- ```
    <EditText
```
- ```
 android:id="@+id/editTextPassword"
```
- ```
        android:layout_width="match_parent"
```
- ```
 android:layout_height="wrap_content"
```
- ```
        android:hint="Contraseña"
```
- ```
 android:inputType="textPassword"
```
- ```
        android:layout_marginTop="10dp" />
```
-
- ```
 <Button
```
- ```
        android:id="@+id/buttonRegistrar"
```
- ```
 android:layout_width="match_parent"
```
- ```
        android:layout_height="wrap_content"
```
- ```
 android:text="Registrar"
```
- ```
        android:layout_marginTop="20dp" />
```
- ```
</LinearLayout>
```

Tiempo: 20 Minutos



# LIVE CODING

## 2. MainActivity.java

```
java
CopiarEditar
package com.ejemplo.formulario;

import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

 EditText nombre, email, password;
 Button registrar;

 @Override
 protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_main);

 nombre = findViewById(R.id.editTextNombre);
 email = findViewById(R.id.editTextEmail);
 password = findViewById(R.id.editTextPassword);
 registrar = findViewById(R.id.buttonRegistrar);
 }
}
```

# LIVE CODING

```
registrar.setOnClickListener(new View.OnClickListener() {
 @Override
 public void onClick(View v) {
 String datos = "Nombre: " + nombre.getText().toString()
 + "\nCorreo: " + email.getText().toString()
 + "\nContraseña: " + password.getText().toString();

 Toast.makeText(MainActivity.this, datos, Toast.LENGTH_SHORT).show();
 }
});
}
```

# LIVE CODING

## Objetivos de aprendizaje

- Uso de `LinearLayout` y atributos como `gravity`, `padding` y `hint`.
- Propiedades de `EditText` (`inputType`, `hint`).
- Evento de clic con un `Button`.
- Mostrar datos con `Toast`.



# Resumen

¿Qué logramos en esta clase?

- ✓ Elementos de interfaz de Android
- ✓ Elementos básico
- ✓ Lenguajes de un proyecto Android
- ✓ Seleccionar lenguajes
- ✓ Vistas Básicas
- ✓ Usos de las vistas
- ✓ Editor de vistas
- ✓ Usos y características
- ✓ Formato de XML
- ✓ Estructura de los XML
- ✓ Layouts
- ✓ Tipos de Layouts

¿Alguna consulta?





## ¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

---

< **¡Muchas gracias!** >

